

- [Introduction à l'environnement .net](#)

- **Base du langage C#**

- [Les commentaires](#)
- [Les variables](#)
- [Types de données](#)
- [Expression](#)
- [Structures de contrôle](#)
- [Les fonctions](#)
- [Les tableaux](#)
- [Chaines de caractères](#)
- [Exercices](#)

- [Les classes](#)
- [L'héritage](#)
- [Compléments de langage](#)
- [Fonctions avancées](#)
- [Projet - Space Invaders](#)
- [_____](#)
- [Git](#)
- [Space Invaders - Piste Verte](#)

•

- Base du langage C#

Base du langage C# 3

C# est un langage introduit par Microsoft en 2000. C'est un langage objet avec un typage statique fort, une syntaxe héritée du C/C++ et une philosophie très proche de Java. C# est un langage phare du framework .net qui se popularise pour la conception de sites Web (ASP), d'ERP (Sharepoint), de scripting et d'applications lourdes.

A l'origine considéré comme une pâle copie de Java, le C# a ensuite bénéficié d'une politique de développement très dynamique par rapport à Java, resté en désérence plusieurs années par Sun avant son rachat par Oracle. Aujourd'hui, C# est un langage de programmation moderne avec une bibliothèque standard très riche et des outils de développement avancés.

La syntaxe du C# est très proche du C et de Java et ne devrait donc pas poser de difficulté si vous avez déjà pratiqué l'un de ces langages. Les éléments de base de la structuration du code sont les accolades `{ }`, qui définissent les blocs, et le point virgule `;`, qui marque la fin d'une instruction. Le saut de ligne est un caractère d'espacement comme un autre (une instruction peut s'étaler sur plusieurs lignes).

Les plus courageux peuvent consulter le [guide officiel de programmation C#](#) et les [spécifications complètes du langage](#).

Les commentaires

Les **commentaires** sont introduits comme en C/C++ et Java :

- sur une seule ligne: tout ce qui suit les caractères `//` est ignoré jusqu'à la fin de la ligne;
- sur plusieurs lignes: tout ce qui se trouve entre les balises `/*` et `*/` est ignoré.

```
// Je suis un commentaire sur une seule ligne.  
/* Je suis un commentaire sur plusieurs lignes.  
   J'apparais grisé dans Les éditeurs offrant une coloration syntaxique */
```

Les variables

Un programme stocke l'information dans des **variables**. Une variable correspond à un espace de la mémoire auquel on donne un nom, on parle d'**identificateur**, et auquel on associe un **type de donné**. Le contenu de cet espace mémoire peut alors être manipulé (lu ou écrit) en le désignant par son identificateur.

Déclaration

Avant de pouvoir utiliser une variable il faut la **déclarer** avec la syntaxe suivante :

Syntaxe
TypeDeDonnee Identificateur; TypeDeDonnee Identificateur = ValeurInitiale;

Exemples :

```
int unEntier;  
float unNombreFloatant = 3.0; // déclaration de la variable unNombreFloatant de type  
float et initialisation  
Bitmap uneImage; // déclaration de la variable uneImage de type Bitmap
```

Avertissement

Une variable peut être déclarée sans être **initialisée** : sa valeur est alors indéfinie. Essayer de lire une variable alors que sa valeur est indéfinie est une erreur de compilation en C#.

Un identificateur est un nom donné par l'utilisateur et il est fortement conseillé de choisir un nom explicite, c'est à dire qui décrit le rôle de la variable. La seule exception à cette règle concerne les variables de boucle pour lesquelles les identificateurs courts `i`, `j`, `k` sont réservés. Ces aspects sont traités en détail dans les [recommandations générales de nommage](#) du guide de programmation C#.

```
int a = 20; // à éviter : impossible de voir à quoi correspond cette variable à partir de  
son nom  
int largeurPixel = 20; // beaucoup mieux: on sait immédiatement que la variable correspond  
à une largeur  
// et que l'on compte en nombre de pixels
```

Les identificateurs ne peuvent pas être des [mots clés](#) du langage (`int`, `if`, `else`...). Ils sont sensibles à la casse (majuscule, minuscule) et doivent obéir à certaines règles : ne pas commencer par un chiffre, ne pas contenir certains signes... ([l'ensemble des règles](#) est plutôt complexe, mais en pratique tenez vous en aux caractères alphanumériques et tout ira bien).

Note

En C#, on suit la **convention Camel Case** : les différents mots constituant un identificateur commencent par une majuscule et ne comportent pas de caractère de séparation. De plus, pour les identificateurs de variables, le premier mot commence par une minuscule. Pour plus d'information sur le sujet, voir [l'utilisation des majuscules](#). dans le guide de programmation.

```
int jeSuisLaNormeCamelCase;
```

Principe du Camel Case (source: wikipedia) [3](#)

Exercice 1 : Identificateurs

Pour chacune des déclarations de variables ci-dessous, dites si l'identificateur choisi semble approprié :

```
int rhs;
```

```
...
```

```
int i;
```

```
...
```

```
double solde_compte;
```

```
...
```

```
double soldeCompte;
```

```
...
```

```
bool lResultat;
```

```
...
```

```
int NombreDeChats;
```

[Corriger](#)

Types de données ³

Le langage C# est

- fortement typé : il est interdit de mettre une valeur dans une variable si les types de la variable et de la valeur ne sont pas compatibles;
- manuellement typé : les informations de typage sont données par le développeur;
- statiquement typé : le typage est vérifié par le compilateur et toute erreur empêche la compilation du programme;
- dynamiquement typé : le CLR vérifie également l'intégrité des types à l'exécution.

Chaque variable est associée à un type donné. Ces types sont divisés en 2 catégories:

1. Les types **valeurs** composés des **types primitifs** (types de base du langage) et des structures (non traitée dans ce cours). Pour cette catégorie, l'emplacement mémoire correspondant à la variable contient directement la valeur concernée.

Type de données	Genre	Taille (octets)	Valeurs possibles
<code>byte</code>	nombre entier	1	à
<code>short</code>	nombre entier	2	à
<code>int</code>	nombre entier	4	à
<code>long</code>	nombre entier	8	à
<code>float</code>	nombre réel	4	à
<code>double</code>	nombre réel	8	à
<code>decimal</code>	nombre réel	16	à
<code>bool</code>	valeur booléenne	1	<code>true</code> ou <code>false</code>
<code>char</code>	caractère	2	<code>U+0000</code> à <code>U+FFFF</code> (caractères Unicode)

2. Les types **références** composés des **classes** définies par l'utilisateur ou présentes dans l'API (en particulier les tableaux). Dans ce cas l'emplacement mémoire correspondant à la variable contient une référence vers un objet. Nous reviendrons en détail sur le comportement de cette catégorie de types.

Constantes littérales [§](#)

Les **constantes littérales**, c'est-à-dire, les valeurs rentrées directement dans le code sont typées :

```
1 // est un entier (Le type exacte byte, short, int ou long sera déterminé par le
contexte)
1.0 // est un double
1.0f // est un float
'c' // est un caractère
true // est un booléen
```

Transtypage

Le **transtypage** ou **cast** désigne l'action de transformer une valeur d'un type de données vers un autre type. Pour caster une valeur dans un nouveau type, il faut indiquer le nom du nouveau type entre parenthèses devant la valeur:

```
double d = 2.1;
int i = (int)d; // d est casté en int, i contient la valeur 2: la partie décimale est supprimée
```

Si le transtypage n'est pas possible, il y aura une erreur, soit au moment de la compilation, soit au cours de l'exécution du programme.

Le C# n'est pas un langage *très* fortement typé: le compilateur accepte de réaliser automatiquement certains transtypages si ces derniers peuvent se faire sans perte. On parle de **conversion implicite**.

```
int x = 2;
double d = x; // 2 est converti en double, le compilateur ne dit rien
int y = d;    // ERREUR, la conversion automatique de double vers int n'est pas possible
```

Parmi les types primitifs, les conversions implicites suivantes sont possibles: `double` `float` `long` `int` `short` `byte`.

Attention

Les opérations de conversions implicites par le compilateur peuvent paraître anodines mais elles sont un élément clef d'un des composants les plus complexes des langages de programmation moderne: la **résolution de nom**. La [résolution de nom](#) consiste, pour le compilateur, à déterminer quel est l'identificateur correcte à utiliser en fonction du contexte.

Ce mécanisme est très complexe dans les langages modernes, chaque couche d'abstraction générant de nouvelles règles (polymorphisme paramétriques, polymorphismes d'héritage, généricité, template, duck typing). Cela mène à des résultats surprenant lorsque l'on ne maîtrise pas bien ces mécanismes. Par exemple, ruby et javascript sont connus pour leurs [conversions implicites hasardeuses](#) alors que les interactions entre conversions implicites et templates de la librairie standard du C++ génère des messages d'erreurs [interminables et difficilement compréhensibles](#).

Exercices

Exercice 2 : Catégories de types

Les 2 catégories de contenu pour les variables en C# sont

[Corriger](#)

Exercice 3 : Typage

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

Le C# utilise du typage statique.

Le C# utilise du typage dynamique.

En C# il n'y a pas de vérification de type à l'exécution du programme.

Un transtypage permet de changer le type d'une variable.

On peut forcer n'importe quelle conversion avec un transtypage explicite.

Le cast d'un `double` en `int` est réalisé avec une opération d'arrondi.

[Corriger](#)

Exercice 4 : Constantes et conversions implicites

Pour chacune des déclarations/initialisations de variables, indiquez si elle est correcte ou fausse:

<code>bool b1 = true;</code>	<code>float f2 = 1.0f;</code>	<code>short s3 = 1.0;</code>
<code>bool b2 = 0;</code>	<code>float f3 = 1.0;</code>	<code>int i1 = 1;</code>
<code>bool b3 = "true";</code>	<code>double d1 = 1;</code>	<code>int i2 = 1.0f;</code>
<code>char c1 = 'b';</code>	<code>double d2 = 1.0f;</code>	<code>int i3 = 1.0;</code>

```
char c2 = 46;
```

```
double d3 = 1.0;
```

```
long l1 = 1;
```

```
char c3 = "a";
```

```
short s1 = 1;
```

```
long l2 = 1.0f;
```

```
float f1 = 1;
```

```
short s2 = 1.0f;
```

```
long l3 = 1.0;
```

[Corriger](#)

Expression ³

Une **expression** est une combinaison de variables, d'opérateurs, de constantes et d'appels de fonction. Comme les variables, les expressions ont une valeur et un type. Par contre les expressions ne sont pas identifiées par un nom. Le calcul de la valeur d'une expression est appelé **évaluation**.

```
int x = 2;  
int y = x + 2; // x + 2 est une expression de type int et de valeur 4  
bool b = y >= 5; // y >= 5 est une expression de type bool et de valeur false
```

Les **règles de priorité** classiques sont utilisées lors de l'évaluation d'une expression (par exemple la multiplication est prioritaire sur l'addition). Les **parenthèses** permettent de changer l'ordre d'évaluation des différentes parties d'une expression et de clarifier la lecture.

Opérateurs simples ³

La [liste des opérateurs du C#](#) est assez longue, les plus communs sont:

- Opérateurs de comparaison : ==, !=, <, >, <=, >=
- Opérateurs logiques : || (ou), && (et), ! (négation)
- Opérateurs numériques: +, -, *, /, % (modulo)

Lorsque les 2 opérandes numériques d'un opérateur binaire ne sont pas du même type, l'opérande du type le plus petit est automatiquement convertie vers le type le plus grand. Par exemple:

```
1.0 + 2 // équivalent à 1.0 + 2.0: 2 est converti en double
```

Opérateur d'affectation

L'**opérateur d'affectation** permet de modifier la valeur d'une variable (à gauche du signe égal) avec une nouvelle valeur (à droite du signe égal). On dit que les variables sont des **left-value** car elles peuvent être mises à gauche d'un signe égal: elles correspondent à une zone de mémoire modifiable.

```
int x;  
x = 4;
```

Important

L'opérateur d'affectation fonctionne toujours par copie. La variable reçoit une copie de la valeur à droite du signe égal.

```
int x = 1;  
int y = 0;  
  
y = x; // y = 1  
y = y + 1; // y = 2, x n'est pas modifié et vaut toujours 1
```

Dans le cas d'un type référence, la valeur stockée dans la variable étant la référence, l'opérateur d'affectation copie cette référence et non l'objet référencé.

Il existe une variante de l'opérateur d'affectation pour chaque opérateur binaire. Par exemple l'opérateur dérivé  permet d'ajouter une valeur à une variable:

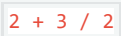
```
int x = 3;  
x += 2; // équivalent à x = x + 2;
```

Remarque: Le type d'une expression d'affectation  est le type de  et sa valeur est la valeur de .

Exercices

Exercice 5 : Expressions numériques

Pour chacune des expressions, donnez son type et sa valeur :

Expression	Type	Valeur
		
		
		

Expression	Type	Valeur
<code>2.0 + 3 / 2</code>		
<code>2 + 3.0 / 2</code>		
<code>5 <= 3 + 2</code>		
<code>1.5f + 2</code>		
<code>1.5f + 2.0</code>		

[Corriger](#)

Structures de contrôle

On retrouve les structures de contrôle classiques : **if**, **else**, **for**, **while**.

if

Syntaxe

```
if ( condition )
{
    // instructions à réaliser si CONDITION vaut true
} else {
    // instructions à réaliser si CONDITION vaut false
}
```

for

Syntaxe

```
for ( initialisation; condition; instructionDIteration )
{
    // bloc d'instructions à réaliser tant que condition vaut true
}
```

Workflow d'exécution d'une boucle for 

while

Syntaxe

```
while ( condition )
{
    // bloc d'instructions à réaliser tant que condition vaut true
}
```

Workflow d'exécution d'une boucle while

Les fonctions

Une **fonction** est une portion de code qui effectue une tâche ou un calcul. Une fonction est désignée par un identificateur qui permet de l'appeler à partir d'autres endroits du programme, c'est-à-dire de demander à exécuter la portion de code qui correspond à la fonction. La fonction peut retourner une valeur, c'est-à-dire **renvoyer un résultat**. Une fonction peut prendre des **arguments** ou **paramètres** qui désignent les données d'entrée sur lesquelles travaille la fonction.

Déclaration

La déclaration d'une fonction se fait en 2 parties :

- On commence par donner son **prototype** qui contient toutes les informations d'identification de la fonction: son nom, ses paramètres, son type de retour...
- Puis on donne son corps entre accolades qui contient les instructions de la fonction.

Syntaxe

```
TypeDeRetour IdentificateurFonction(TypeParametre1 identificateurParametre1, TypeParametre2  
identificateurParametre2, ...) // prototype  
{  
    // début du corps  
    // instructions de la fonction  
    return valeurDeRetour;  
} // fin du corps
```

Une fonction qui ne retourne pas de résultat utilise le type de retour special `void` (qui signifie rien). Les fonctions qui ne retournent rien sont appelées **procédures**.

Syntaxe

```
void IdentificateurFonction(TypeParametre1 identificateurParametre1, TypeParametre2
identificateurParametre2, ...)
{
    // intructions de la fonction
    return; // optionnel, remarquez l'absence de valeur après l'instruction return
}
```

Attention

Les fonctions dont le type de retour est différent de `void` doivent se terminer par l'exécution d'une instruction `return` suivie d'une valeur dont le type est compatible avec le type de retour de la fonction. Le compilateur vérifie que, quelque soient les paramètres d'entrée de la fonction, l'exécution de la fonction se termine toujours par une instruction `return`. Ainsi le code suivant est faux et provoque le message d'erreur : *Tous les chemins de code ne retournent pas nécessairement une valeur* :

```
int F3(int x)
{
    if(x<2)
    {
        return 1;
    }
}
```

Note

Par convention, les identificateurs de fonction en C# suivent la norme Camel Case mais, contrairement aux identificateurs de variable, on fera commencer leur premier mot par une majuscule.

Appel de fonction

Une fonction est appelée en indiquant son identificateur suivi, entre parenthèses, des valeurs des paramètres:

Syntaxe

```
IdentificateurFonction(valeurParametre1, valeurParametre2,...);
```

Un **appel de fonction** dont le type de retour est différent de `void` est une expression qui a pour type le type de retour de la fonction et pour valeur la valeur retournée par la fonction.

```
int Fonction2(int a)
{
    a += 1;
    return a;
}

void Fonction1()
{
    int b = 7;
    b = Fonction2(b); // Fonction2(b) est une expression de type int et de valeur 8
}
```

Important

Lors de l'appel d'une fonction, le passage des paramètres est toujours réalisé par copie : la fonction reçoit une copie de la valeur passée en paramètre. Si la variable est de type valeur, il s'agit d'une copie de cette valeur. Si la variable est de type référence, il s'agit d'une copie de la référence.

```
void Increment(int a)
{
    a += 1;
}

...

int x = 0;
Increment(x);
int y = x; // y vaut 0 !
```

Portée des variables [§](#)

Un **bloc** est une portion de code délimitée par une paire d'accolades : `{ }`. Les blocs sont organisés de manières hiérarchiques : chaque bloc peut contenir d'autres blocs et ainsi de suite. On parle de **bloc de niveau supérieur** (le contenant) et de **bloc de niveau inférieur** (le contenu).

Les **variables** dites **locales** sont :

- les paramètres d'une fonction,
- les variables déclarées à l'intérieur d'une fonction,
- les variables déclarées dans une boucle.

Une variable locale a une **portée d'utilisation** limitée. Elle ne peut donc pas être utilisée à l'extérieur de sa portée, d'où le caractère local. Sa portée commence à sa déclaration/définition. Pour un paramètre de fonction ou une variable déclarée dans un bloc, la portée se termine à la fin de son bloc : lors de l'accolade fermante. Dans le cas d'une boucle, la variable cesse d'exister lorsque la boucle se termine. Exemple:

```
int Fonction2(int a)
{
    a += 1;
    return a;
} // a cesse d'exister ici

void Fonction1()
{
    int b = 7;
    for(int i=0; i < 10; i++)
    {
        b = Fonction2(b);
    } // i cesse d'exister ici
} // b cesse d'exister ici
```

Exercices

Exercice 6 : Fonctions

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

Toutes les fonctions doivent se terminer par une instruction `return`.

Une méthode ayant pour type de retour `void` est appelée une procédure.

Lors d'un appel de fonction, la fonction appelée reçoit une copie des valeurs passées en paramètre.

[Corriger](#)

Exercice 7 : Portée des variables

Le code ci-dessous contient 3 erreurs (on suppose que la méthode `Afficher` prenant un entier en paramètre existe).

```
void Toto(int a, int b)
{
    int z = 7;
    for (int i = 0; i < b; i++)
    {
        int d = 11;
        Afficher(d + a);
    }
    for (i = 0; i < b; i++)
    {
        int e = 11;
        Afficher(e + d);
    }
    Afficher(a);
    Afficher(z);
    Afficher(e);
}
```

Les numéros de lignes contenant les erreurs sont

[Corriger](#)

Les tableaux

En utilisant un **type existant**, le langage permet de créer des **tableaux**. Un tableau peut être vu comme une liste, un vecteur ou une matrice d'éléments du même type.

Chaque **case** ou **cellule** d'un tableau stocke un élément du type donné.

Déclaration et création

Etant donné un type `T`, la notation `T[]` désigne le nouveau type **tableau d'éléments de type T**. Par exemple `int[]` désigne le type *tableau d'entiers* qui est différent du type *entier*. On peut déclarer des variables avec ce type de données.

Pour créer un tableau, on utilise l'opérateur `new`. Les tableaux sont de **type référence**.

Tableaux unidimensionnels:

Syntaxe

```
TypeDesElements [ ] monTableau1; // déclaration d'un tableau
TypeDesElements [ ] monTableau2 = new TypeDesElements [tailleDuTableau]; // déclaration et création d'un tableau
int [ ] monTableau3 = {1, 5, 6, 7}; // déclaration d'un tableau et initialisation explicite
```

Tableaux multidimensionnels :

Syntaxe

```
TypeDesElements [,] monTableau1; // déclaration d'un tableau bidimensionnel
TypeDesElements [,,] monTableau2; // déclaration d'un tableau tridimensionnel
TypeDesElements [,] monTableau3 = new TypeDesElements [nombreDeLignes, nombreDeColonnes] ;
// déclaration et initialisation d'un tableau
int [,] monTableau4 = { {1, 5, 6},
                       {2, 9, 7} }; // déclaration d'un tableau 2d et initialisation explicite
```

Les variables de type tableau sont comme les autres : elles doivent être initialisées avant d'être utilisées. Lors de la création d'un tableau, chaque cellule du nouveau tableau est **initialisée par défaut**. Ainsi, si vous créez un tableau d'entiers, chaque case sera mise à zéro.

Accès aux éléments d'un tableau

Les cases d'un tableau sont numérotées de 0 à la taille du tableau moins un. Le numéro d'une case est appelé son **indice** et permet d'accéder à cette case.

Syntaxe

```
int [] monTableau1 = {1, 2, 3};  
monTableau1[0] = 4; //modification de la valeur de la case d'indice 0: monTableau1 = {4, 2, 3}  
int a = monTableau1[2]; // lecture de la valeur de la dernière case du tableau: a = 3  
  
int [,] monTableau2 = { {1, 2, 3},  
                        {4, 5, 6}};  
monTableau2[0, 0] = 7; // monTableau2 = { {7, 2, 3}, {4, 5, 6}}  
int b = monTableau2[1, 2]; // b = 6
```

L'accès à un indice invalide d'un tableau (en dehors d'un tableau) déclenche une erreur à l'exécution.

Taille d'un tableau

Pour connaître la taille d'un tableau unidimensionnel on utilise la propriété `Length`:

```
int [] monTableau1 = {1, 2, 3};  
int taille = monTableau1.Length; // taille = 3
```

Pour connaître la taille d'un tableau bidimensionnel on utilise la méthode `GetLength(dimension)`:

```
int [,] monTableau1 = { {1, 2, 3},  
                        {4, 5, 6} };  
int lignes = monTableau1.GetLength(0); // 2  
int colonnes = monTableau1.GetLength(1); // 3
```

Parcours des tableaux

Avec une boucle **for** ou une boucle **foreach** (si les indices des éléments ne nous intéressent pas).

```
bool [] monTableau = {true, true, false};  
  
for (int i = 0; i < monTableau.Length; i++)  
{  
    ... monTableau[i] ...  
}  
  
foreach (bool valeur in monTableau) // se lit: "pour toute valeur de type bool dans monTableau"  
{  
    ... valeur ...  
}
```

Exercices

Exercice 8 : Plateforme .net

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

.....

Les cases d'un tableau sont initialisées par défaut lors de la création du tableau.

.....

Une variable de type tableau qui n'a pas été initialisée contient par défaut un tableau de taille 0.

.....

Si je passe un tableau en paramètre d'une fonction, le contenu du tableau est copié dans la fonction appelée.

.....

Je peux créer un tableau dont la taille est donnée par une variable.

Corriger

Exercice 9 : Déroulement de code

Soit la fonction:

```
int F1()
{
    int[] t = {4, 5, 2};
    t[2] += t[1] + 4;
    int a = 1;
    int b = t[a];
    return t[0] + b + t[2];
}
```

La fonction retourne la valeur:

Corriger

Exercice 10 : Déroulement de code

Soit la fonction:

```
int F2()
{
    int[] t = {4, 1, 3};
    int[] s = {2, 1, 0};
    int a = t[s[1]];
    int b = t[s[0]];
    int c = t[s[s[2]]];
    return a + b + c;
}
```

La fonction retourne la valeur:

[Corriger](#)

Exercice 11 : Déroulement de code

Soit la fonction:

```
void F4()
{
    int[,] t = new int[3,3];
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            t[i,j] = i + j + j * i;
}
```

Le contenu du tableau `t` à la fin de la fonction `F4` est (utilisez le format standard pour l'initialisation des tableaux `{ {....}, {....} }`)

[Corriger](#)

Exercice 12 : Tableaux et type référence

Soit le code:

```
void Gi(int i) { i = 2; }

void Hi(int i) { i++; }

void Gt(int[] t){
    t = new int[2];
    t[0] = 1;
}

void Ht(int[] t){ t[0]++; }

void Test()
{
    int i1 = 0;
    int i2 = 0;
    int [] t1 = {0, 0};
    int [] t2 = {0, 0};

    Gi(i1);
    Hi(i2);

    Gt(t1);
    Ht(t2);
}
```

La valeur de `i1` à la fin de la fonction `Test` est

La valeur de `i2` à la fin de la fonction `Test` est

La valeur de `t1` à la fin de la fonction `Test` est (en utilisant le format standard `{....}`)

La valeur de `t2` à la fin de la fonction `Test` est (en utilisant le format standard `{....}`)

[Corriger](#)

Chaines de caractères [↗](#)

Les chaînes de caractères sont désignées par le type `String` (ou `string` avec une minuscule) qui est de **type référence**. Les constantes de type chaîne de caractères s'écrivent entre **double quotes** `"`.

```
string s1 = "Bonjour";  
String s2 = "Bonjour";  
String s3 = ""; // chaîne de caractères vide
```

L'**opérateur de concaténation** `+` permet de mettre 2 chaînes de caractères bout à bout. Tous les types de données sont implicitement convertibles en chaînes de caractères avec l'opérateur `+`. Ainsi, une expression de la forme « chaîne de caractères » + *n'importe quoi* est toujours valide et son résultat est une chaîne de caractères.

```
double d = 42.0;  
bool b = true;  
  
String s1 = "La valeur de d est " + d; // s1 = "La valeur de d est 42"  
String s2 = "" + b; // s2 = "true"
```

Une chaîne de caractères est composée de caractères au format unicode sur 2 octets (UTF16). Le backslash `\` sert de caractère d'échappement pour obtenir les caractères spéciaux : par exemple `\n` pour le retour à la ligne, `\t` pour une tabulation, `\\` pour obtenir un backslash.

Le caractère `@` placée devant une constante de type `String` désactive l'interprétation des caractères spéciaux. Cela est particulièrement pratique pour les chemins d'accès aux fichiers :

```
String filename = @"C:\dossier\fichier1.txt" ;
```

L'API .net propose de nombreuses méthodes pour [manipuler les chaînes de caractères](#) : (`Replace`, `Substring`, `ToLower`, `ToUpper`, `Insert` ...), test sur le contenu (`IsEmpty`, `Compare`, `Contains`, `Equals` ...), longueur de la chaîne (`Length`), conversion vers les types de données primitifs (ex: `ToInt32`).

Attention

Les chaînes de caractères sont **immuables**: on ne peut pas modifier une chaîne après sa création. Toutes les opérations sur les chaînes de caractères entraînent la création d'une nouvelle chaîne de caractères. Par exemple l'initialisation de la variable `s` dans le code ci-dessous :

```
int i = 1;
double d = 2;
bool b = false;

String s = " i = " + i + ", d = " + d + ", b = " + b + "."; // s = " i = 1, d = 2, b = false."
```

implique un total de 13 chaînes de caractères:

- les 4 constantes entre double quote;
- les 3 chaînes issues des conversions implicites de `i`, `d` et `b`;
- les 6 chaînes créées par chacun des opérateurs `+`.

Exercices

Exercice 13 : QCM les chaînes de caractères

Le type `String` est de type référence.

Pour modifier le 2ème caractère d'une chaîne de caractère `s`, je peux écrire `s[1] = 'a';`.

Pour stocker n caractères j'ai besoin de n octets.

Tous les types de données peuvent être converti en chaîne de caractères.

[Corriger](#)

Exercice 14 : String immutable

Soit la fonction:

```
public String StringifyArray(int [] a){
    String r = "{";
    for(int i = 0; i < a.Length-1; i++)
    {
        r = r + a[i] + ", ";
    }
    if(a.Length>0)
    {
        r = r + a[a.Length-1];
    }
    r = r + "}";
    return r;
}
```

Cette fonction appelée avec en paramètre le tableau d'entier contenant les valeurs 0, 1 et 2 retourne la chaîne de caractère

Cette fonction appelée avec en paramètre le tableau d'entier contenant les valeurs 0, 1 et 2 va créer chaînes de caractères (On ne comptera pas les chaînes constantes données entre double quote, pensez aux conversions implicites et aux opérateurs `+`).

Cette fonction appelée avec en paramètre un tableau d'entier contenant $n > 0$ éléments va créer chaînes de caractères (On ne comptera pas les chaînes constantes données entre double quote).

[Corriger](#)

Exercices

Exercice 15 : Debugage

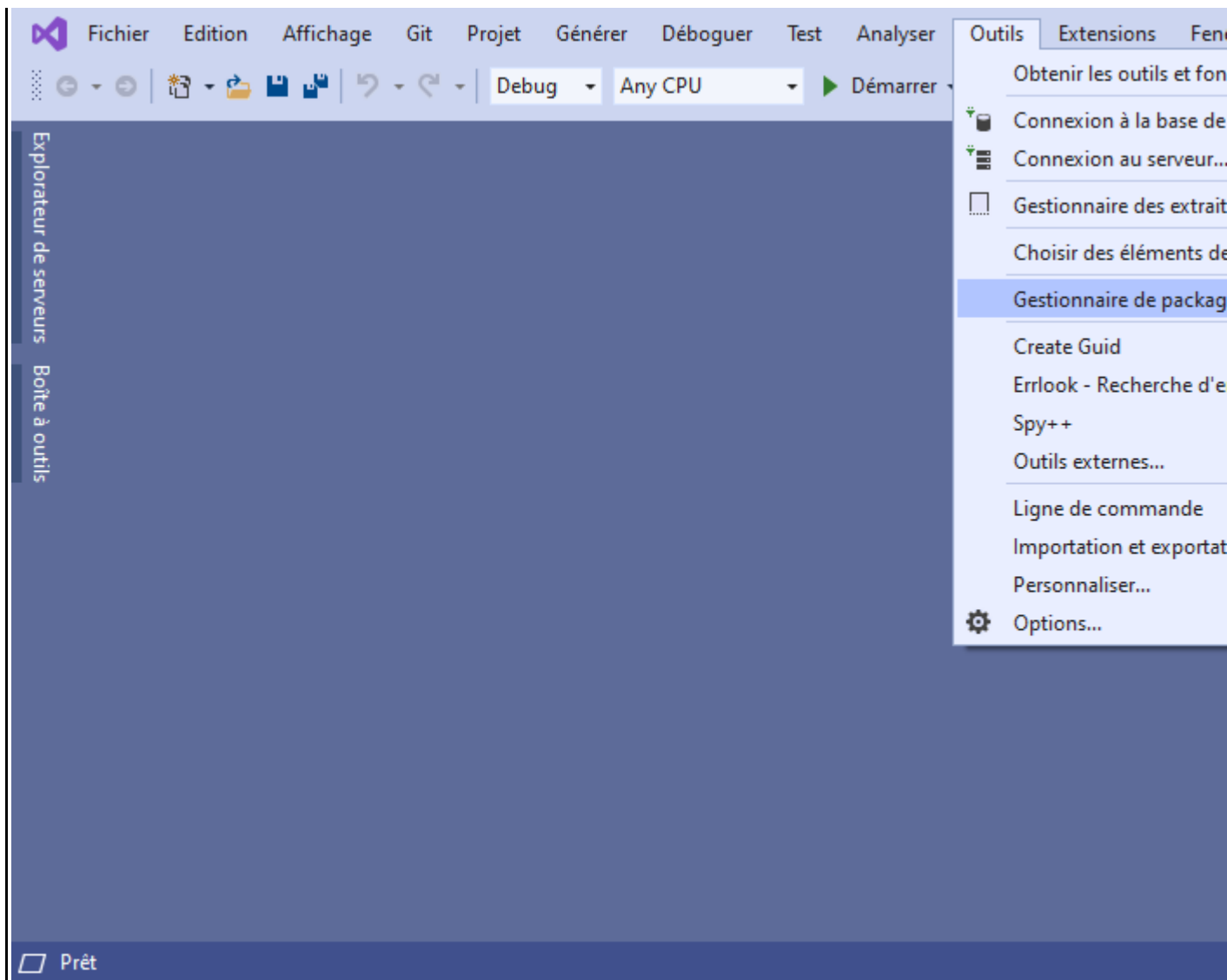
Visual Studio possède un débogueur intégré qui va vous aider dans ce genre de situation. Lorsque l'on compile un projet sous Visual Studio, on peut choisir le mode **Debug** ou **Release**. Le mode Release sert à compiler le programme avant sa diffusion à l'utilisateur: toutes les optimisations seront utilisées par le compilateur pour que le programme soit le plus rapide possible (quitte à réécrire

des bouts de votre code afin de le rendre plus efficace). En mode Debug, le compilateur va au contraire produire un programme dont les instructions collent exactement au code que vous avez écrit afin de pouvoir suivre son exécution pas à pas. Normalement, durant la phase de développement on compile en mode Debug.

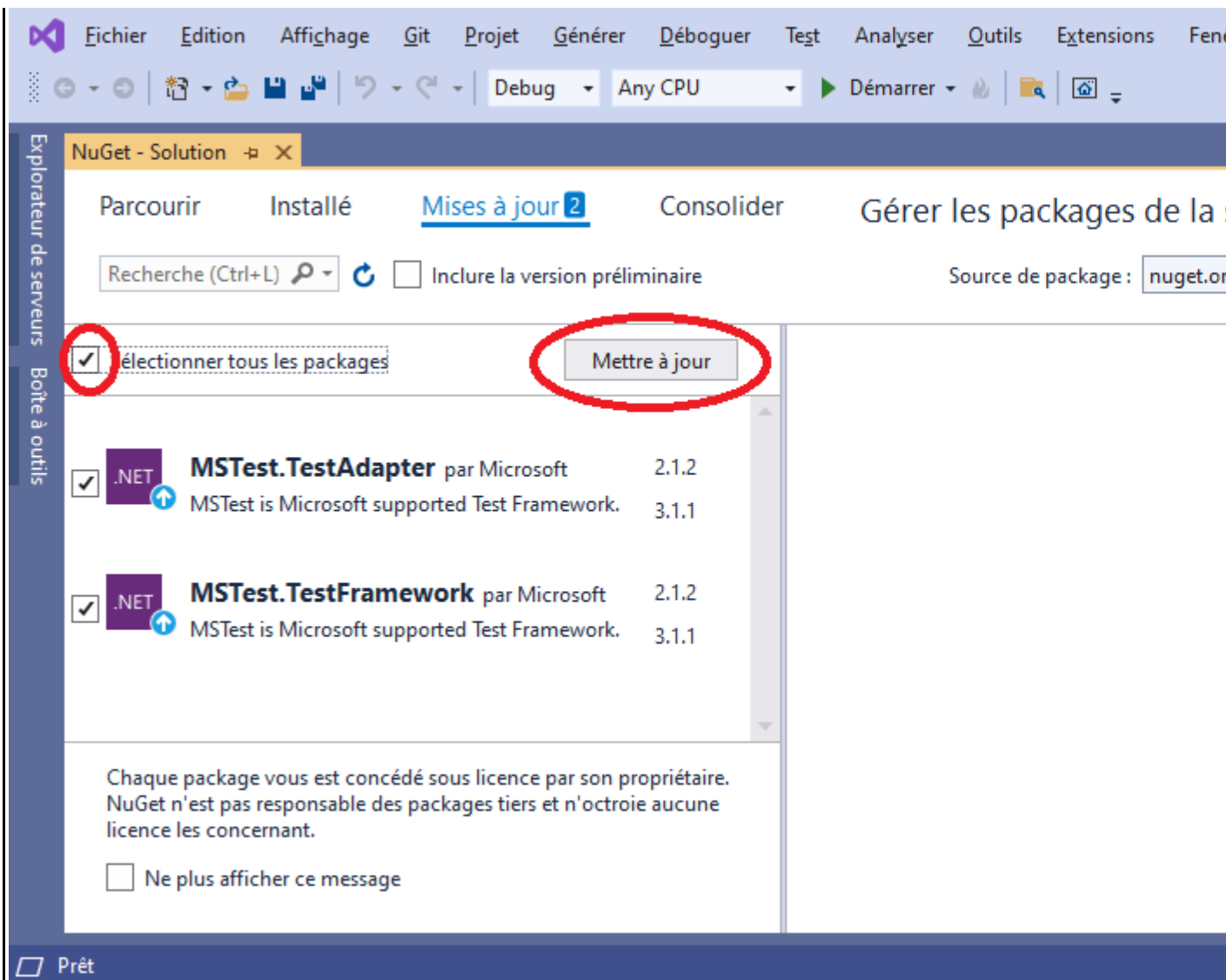
Téléchargez la [solution Visual Studio](#), décompressez l'archive, ouvrez la solution, ouvrez le fichier *Program.cs* et laissez vous guider par les commentaires !

Exercice 16 : Manipulation de tableaux

Pour cet exercice, commencez par télécharger la [solution Visual Studio](#), décompressez l'archive et ouvrez la solution. Ce projet comporte des tests unitaires; commencez par mettre à jour les plugins Nugets de Visual Studio qui gèrent les tests. Allez dans le menu « Outils -> Gestionnaire de Package NuGet-> Gérer les packages NuGet... »

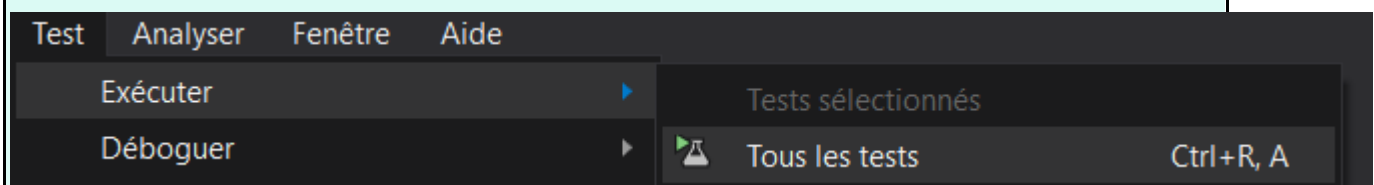


Cliquez sur l'onglet « Mises à jour », sélectionnez tous les packages, puis cliquez sur le bouton « Mettre à jour ». Acceptez les conditions et attendez la fin du processus de mise à jour.



Important

A chaque fois que vous écrivez une fonction, exécutez les tests unitaires (menu *Test* -> *Exécuter* -> *Tous les tests* ou touches `Ctrl+R Ctrl+A`). Une fonction n'est pas correcte tant qu'il y a un test qui échoue sur la fonction.



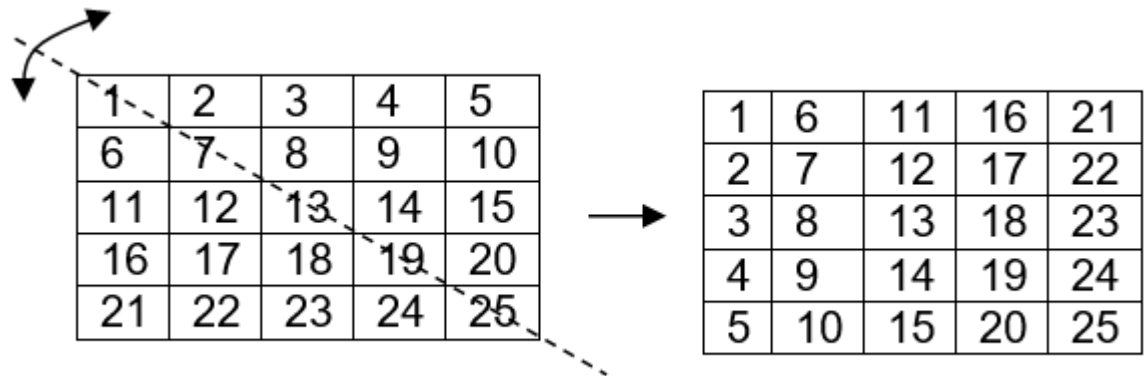
- Complétez la fonction `InsererTableau` recevant en paramètre deux tableaux d'entiers `origine` et `aInserer` ainsi qu'un indice `position` compris entre 0 et la taille du tableau `origine`. Cette fonction retourne un nouveau tableau d'entiers correspondant au tableau `origine` où l'ensemble des valeurs de `aInserer` ont été insérées à l'élément d'indice `position`. Exemple:

```

• int [] t1 = {4, 5, 3, 8, 9, 1, 2};
• int [] t2 = {11, 15, 13};
• int [] t3 = InsérerTableau(t1, t2, 2);
• // t3 = {4, 5, 11, 15, 13, 3, 8, 9, 1, 2}

```

- Complétez la fonction `TransposerMatrice` permettant d'effectuer la symétrie d'un tableau de nombres réels bidimensionnel carré de taille n:



- Complétez la fonction `RemplirRectangle` recevant en paramètre un tableau d'entiers bidimensionnel, 2 coordonnées entières `x` et `y` et 2 dimensions entières `largeur` et `hauteur`. Cette fonction dessine un rectangle dans le tableau en écrivant des 1 dans les cases. Le coin supérieur gauche du rectangle est donné par les coordonnées `(x,y)`, les dimensions du rectangle sont données par `largeur` et `hauteur`. Attention: le rectangle peut sortir du tableau !

```

• int [] t = new int[5,7];
• RemplirRectangle(t, 3, 2, 5, 2);
• /*
• t = {{0, 0, 0, 0, 0, 0, 0},
•      {0, 0, 0, 0, 0, 0, 0},
•      {0, 0, 0, 1, 1, 1, 1},
•      {0, 0, 0, 1, 1, 1, 1},
•      {0, 0, 0, 0, 0, 0, 0}}
• */

```

© Droits d'auteur B. Perret, ESIEE Paris 2022.

Compilé avec [Sphinx](#) en utilisant un [thème](#) fourni par [Read the Docs](#).

- [Base du langage C#](#)

- **[Les classes](#)**

- [Une extension de la notion de type](#)
- [Champs et méthodes d'instance](#)
- [Création d'un objet](#)
- [Opérateur d'accès](#)
- [Les constructeurs](#)
- [Encapsulation](#)
- [Membres statiques](#)
- [Exercices](#)

- [L'héritage](#)
- [Compléments de langage](#)
- [Fonctions avancées](#)
- [Projet - Space Invaders](#)
- [Git](#)
- [Space Invaders - Piste Verte](#)

-

- Les classes

Les classes ³

Une extension de la notion de type ³

Un **type de données**, ou simplement **type**, désigne la conjugaison de deux aspects fondamentaux de la programmation :

1. un regroupement de données, on parle **d'agrégation**, et,
2. des méthodes pour initialiser ces données et les manipuler.

Par exemple le type `int` décrit comment stocker un nombre entier signé sur 32 bits (les données) ainsi que les opérateurs communs pour manipuler ces nombres +, *, - ... (les méthodes). Notez que le type `int`, n'existe pas en mémoire, c'est la variable de type `int` qui va déclencher la réservation d'une case mémoire pour son stockage. Le type `int` est unique mais permet la création de plusieurs variables de type `int`.

En dehors des types de base comme les types numériques, il vous est possible de créer ou d'utiliser d'autres types de données à l'intérieur de vos programmes. Ces types plus complexes seront désignés sous le terme de **classes**. Dit autrement, une classe est une description d'un nouveau type de données. Tout comme pour le type `int`, une classe est unique mais permet de **créer** (ou **instancier**), plusieurs **objets** (ou **instances**).

Attention

La classe et l'objet sont des concepts liés mais intrinsèquement différents. Par exemple, prenons un objet réel comme une voiture. Le modèle numérique de la voiture décrit sa forme, ses fonctions, comment elle doit être fabriquée. Ce modèle n'est pas une voiture. De la même manière une classe n'est pas un objet.



René Magritte « La Trahison des Images » [🔗](#)

Il y a 1 type pour 0 à n objets créés. A partir d'un plan, plusieurs pièces peuvent être créées, ou aucune aussi. Les pièces ainsi créées partagent les informations décrites dans le plan. Les pièces existent indépendamment. Une fois la pièce A intégrée dans une voiture, elle n'est plus en relation avec la pièce B assemblée dans un avion. Ces deux pièces sont identiques au sens où elles sont deux pièces du même modèle. Cependant elles sont différentes au sens où elles vont vivre indépendamment. Ainsi la pièce B pourra être remplacée car cassée au bout d'un an alors que l'autre pourra vivre dix ans.

Les objets peuvent avoir des attributs propres, dans ce cas, ses attributs doivent être décrits dans le plan. Par exemple, un autoradio contient une liste de stations préférées configurées par l'utilisateur. Le modèle de l'autoradio indique combien de canaux préférés peuvent être choisis cependant aucun n'est établi avant que l'utilisateur ne les choisissent. De même, une voiture C3 est éditée suivant plusieurs couleurs de carrosserie et plusieurs types de moteurs. Pourtant une C3 noire (on parle ici d'un objet) et une C3 blanche (on parle toujours d'un objet) reste des voitures du modèle C3. Il en va de même pour une C3 diesel et une C3 essence.

Note

Nous dirons parfois *un objet cercle* au lieu d'*un objet de type Cercle*.

La déclaration d'une nouvelle classe se fait grâce au mot clef **class** suivi d'un identificateur : le nom de la nouvelle classe. Le corp de la classe (son contenu) est placé entre accolades. En général, une classe *A* est déclarée directement dans un fichier portant le même nom *A.cs* (une classe par fichier). Par convention, les identificateur de classe utiliseront également la norme CamelCase avec la premier lettre en majuscule.

Syntaxe

```
class Identificateur{  
    // corps de la classe  
}
```

Exercice 1 : Instanciation

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse :

- -
On ne peut instancier qu'un objet à partir d'une classe.
- -
Une classe décrit un type de données.
- -
Une classe permet de créer plusieurs objets de même caractéristiques.
- -
Une classe peut porter plusieurs noms différents.

Corriger

Note

Les recommandations de nommage pour les classes sont traitées [ici](#) .

Champs et méthodes d'instance

Une variable propre à un objet (la couleur, l'immatriculation de la voiture) est qualifiée de **champ d'instance** (ou **variable d'instance**, ou **attribut** ou **instance field**). Entre deux objets de type *Voiture*, ces paramètres peuvent avoir des valeurs différentes. Une fonction propre à un objet est qualifiée de **méthode d'instance** (**method**). Cette méthode doit être appelée depuis une instance particulière et n'influe que sur le comportement de cet objet : les différentes instances d'une même classe sont indépendantes.

Les champs et les méthodes d'instances d'une classe sont appelés les **membres d'instance** (**instance members**) de la classe. Chaque membre d'instance est précédé d'un **qualificatif de visibilité** qui donne les droits d'accès depuis *l'extérieur*, c'est-à-dire depuis du code ne se trouvant pas la classe courante (celle que l'on est en train d'écrire). Le qualificatif **public** ouvre l'accès sans restriction. Pour le moment, on utilisera le qualificatif public mais nous reviendrons là-dessus dans le chapitre **Encapsulation**.

La déclaration de champs et de méthodes d'instance se fait à l'intérieur de la classe:

Syntaxe

```
class Identificateur{

    // déclaration des champs d'instances
    public TypeChamp1 identificateurChamp1;
    public TypeChamp2 identificateurChamp2;
    ...

    // déclaration des méthodes d'instances
    public TypeDeRetour1 IdentificateurMethode1(TypeParametre11
identificateurParametre11, ...) {...}
    public TypeDeRetour2 IdentificateurMethode2(TypeParametre21
identificateurParametre21, ...) {...}
    ...
}
```

Exemple:

```
class Voiture{

    public int couleur ;
    public int puissance ;

    public void Demarrer() { ... }
    public void Arrêter() { ... }
}
```

L'ordre de déclaration des membres d'une classe n'a pas d'importance. Par convention, on déclarera les champs d'instance en premier, puis les méthodes.

Exercice 2 : Membres d'instance

- -
Tous les objets d'une même classe possèdent les mêmes champs et les mêmes méthodes d'instance.
- -
Tous les champs des objets d'une même classe possèdent des valeurs identiques.
- -
Lorsque je déclare un champ d'instance, le CLR va réserver de la mémoire pour ce champs au lancement du programme.
- -
Variable d'instance est un synonyme de champ d'instance.
- -
Les champs et les méthodes d'instance font partis des membres d'instance d'une classe.
- -
Une classe doit contenir des champs d'instance.
- -
Une classe doit contenir des méthodes d'instance.

Corriger

Afin de facilement identifier le rôle d'un membre d'instance, on pourra s'en tenir aux [conventions de nommage](#) données dans le guide de programmation C#.

Création d'un objet³

La **création** d'un objet, ou **instanciation**, est réalisé avec l'opérateur **new** qui retourne une **référence** vers le nouvel objet créé.

Syntaxe

```
identificateurVariable = new IdentificateurClasse(parametre1, parametre2, ...);
```

Il y a deux actions effectuées :

1. Création d'un objet de type *IdentificateurClasse* en utilisant les paramètres éventuels : c'est la partie à droite du signe égale;
2. Initialisation de la variable *identificateurVariable* avec la référence retournée par l'opérateur `new`.

Exemple:

```
/* déclaration d'une variable de type Bitmap
   et initialisation avec la référence vers un nouvel objet de type Bitmap avec les
   paramètres (400, 200)*/
Bitmap image = new Bitmap(400,200);
/* déclaration d'une variable de type Voiture
   et initialisation avec une référence vers un nouvel objet de type Voiture */
Voiture voiture = new Voiture();
```

Attention

Les classes font parties de la catégorie **type référence**. Cela signifie qu'une variable dont le type correspond à une classe ne contient pas directement un objet mais une référence vers un objet. Ce lien de *référence* est généralement représenté par une flèche et on dit que la variable *référence* l'objet ou *pointe vers* l'objet (en référence à la notion de pointeur en C/C++).

Par abus de langage, une variable de catégorie **type référence** sera simplement appelée **référence**. Ainsi, dans l'exemple précédent que `image` est une référence de type `Bitmap`.

Considérons le code :

```
class C{
    public int i;
    public bool b;
}

...

int v = 2;
C c = new C();
```

La mémoire contient 2 variables, `v` et `c`, et un objet de type `C`. Elle peut être représentée ainsi:

Représentation schématique de l'état de la mémoire du programme. [↗](#)

En pratique, la mémoire de l'ordinateur est divisée en octets. Chaque octet reçoit un numéro d'identification unique allant de 0 jusqu'à la taille maximale physiquement disponible (la mémoire peut être vue comme un tableau d'octets). Ce numéro est l'adresse en mémoire. L'adresse identifie ainsi un endroit unique dans la mémoire. Pour stocker un objet, le système réserve un bloc de mémoire. Ainsi, un objet de 10 octets peut occuper les cellules contigües 80, 81, 82... 89. La référence va contenir l'adresse de la première cellule soit la valeur 80. Durant les exercices, plutôt que de créer des adresses fictives, il est plus simple de schématiser la référence comme une flèche vers une autre zone de la mémoire.

Exercice 3 : Référence et création d'instances

- -
Si je crée trois références alors elles désignent trois objets différents.
- -
Il peut exister plusieurs variables qui référencent un même objet.
- -
Il existe trois catégories de variables : valeur, référence et objet.
- -
La déclaration de variable `A v;` ou `A` est un identificateur de classe ne provoque pas la création d'un objet de type `A`.
- -
Pour créer un nouvel objet à partir d'un objet existant, je déclare une variable que j'initialise avec une référence vers cet objet.
- -
L'instruction `Voiture v = v;` est-elle valide ?

-

Les instructions `Voiture v = new Voiture(); v=v;` sont-elles valides ?

[Corriger](#)

Opérateur d'accès [↗](#)

L'**accès aux membres d'instances**, champs et méthodes, se fait à travers l'utilisation d'une référence sur l'objet concerné et de l'**opérateur d'accès point (dot)** `.`

Syntaxe

```
// accès à un champ d'instance  
reference.identificateurChamps
```

```
// appel d'une methode d'instance  
reference.IdentificateurMethode(parametre1, parametre2, ...)
```

Exemple:

```
Voiture referenceVoiture = new Voiture();  
referenceVoiture.Demarre();  
int p = referenceVoiture.puissance;
```

Une référence qui contient la valeur spéciale `null` ne désigne aucun objet. Tenter d'appliquer l'opérateur d'accès `.` sur une référence `null` est une erreur de programmation qui provoque un crash du programme à l'exécution.

Instance courante [↗](#)

Nous venons de voir que toutes les méthodes d'instance sont appelées à partir d'une référence (à gauche de l'opérateur `.`). Dans la méthode d'instance, cette référence est appelée **instance courante** et est désignée par le mot clé `this`. L'instance courante est une référence sur l'objet sur lequel on est en train de travailler.

Exemple:

```
class Compteur  
{  
    public int valeur;  
  
    public void SetValeur(int nouvelleValeur)  
    {
```

```

        this.valeur = nouvelleValeur; // this.valeur désigne le champ valeur de
l'objet sur lequel la méthode SetValeur est appelée
    }

Compteur compteur = new Compteur();
compteur.SetValeur(10);

```

Un objet de type `Compteur` est créé. Cet objet est référencé par la variable `compteur`. L'opérateur d'accès est appliqué sur la référence `compteur`. Il permet ainsi d'accéder à la fonction `SetValeur()` de l'objet concerné. L'opérateur d'accès fixe l'objet référencé comme instance courante lors de l'exécution de la fonction `SetValeur()` (`this` est égal à `compteur`).

Afin de simplifier l'écriture, on peut accéder aux membres d'instance de l'instance courante sans utiliser la référence `this` et l'opérateur `.`. Si on ne précise pas de référence avec l'opérateur `.` devant un nom d'identificateur qui correspond à un membre d'instance, le compilateur ajoutera automatiquement `this.` devant l'identificateur. Le code suivant est donc équivalent au précédent:

```

class Compteur
{
    public int valeur;

    public void SetValeur(int nouvelleValeur)
    {
        valeur = nouvelleValeur; // équivalent à this.valeur = ...
    }
}

```

Note

A l'intérieur du code d'une méthode d'instance, l'accès aux membres de l'instance courante peut donc se faire directement sans passer par l'opérateur d'accès. Les normes d'écriture privilégient généralement cette facilité. Une exception notable apparaît lorsqu'il est nécessaire de faire la différence entre un champ d'instance et une variable locale ayant le même identificateur.

```

class Compteur
{
    public int valeur;

    public void SetValeur(int valeur) // Le paramètre a le même nom que le champ
    {
        this.valeur = valeur; // this.valeur désigne le champ d'instance, alors que
valeur désigne la variable locale
    }
}

```

Indépendance entre objets ³

Si l'on crée deux instances de la classe `Compteur` :

```
Compteur c1 = new Compteur();  
Compteur c2 = new Compteur();
```

Deux zones mémoires sont réservées pour stocker chaque instance. Deux références `c1` et `c2` sont créées et associées à chaque objet. On peut représenter la mémoire de la façon suivante :

Si on exécute l'instruction:

```
c1.SetValeur(10);
```

L'opérateur d'accès `.` appliqué sur la référence `c1` m'indique que l'instance courante `this` va être l'objet référencé par `c1`. Ainsi, c'est le champ `valeur` de cet objet qui va être modifié et changé pour la valeur 10. Le 2ème objet ne subit aucune modification et la mémoire devient:

Exercices

Exercice 4 : Membres d'instances

- -
En modifiant une variable `v` de type référence, je ne modifie pas l'objet référencé par `v`.
- -
Si je change la valeur d'un champ d'instance, l'objet n'est plus considéré comme appartenant à sa classe d'origine.
- -
Une fois l'objet créé et ses variables d'instance initialisées, je ne peux plus les modifier.
- -
Je peux accéder directement à un objet sans passer par une référence.

- -
Si j'appelle une méthode depuis une référence et que cette référence a pour valeur `null` alors le programme plante.
- -
Soient deux références `r1` et `r2` qui désignent le même objet, si je change `r1` pour qu'elle réfère un autre objet, alors `r2` désigne aussi ce nouvel objet.
- -
Si aucune référence ne désigne un objet, alors je ne peux pas accéder aux attributs de cet objet.
- -
Dans une méthode d'instance, `this` est toujours une référence vers un objet de la classe courante.
- -
Dans une méthode d'instance, `this.v` et `v` peuvent désigner 2 choses différentes.

Corriger

Les constructeurs [3](#)

Chaque classe contient un ou plusieurs **constructeurs**. Cette fonction particulière est appelée lors de la création d'un objet par l'opérateur `new`. Le but du constructeur est d'initialiser les champs d'instances de l'objet. Un constructeur est déclaré comme une méthode sans type de retour et dont le nom est identique à celui de la classe :

Syntaxe

```
public IdentificateurDeClasse(TypeParametre1 identificateurParametre1, TypeParametre2  
identificateurParametre2, ...)  
{...}
```

Exemple :

```
class Cercle
{
    public int x;
    public int y;

    public Cercle() // constructeur sans argument
    {
        x = 10;
        y = 10;
    }

    public Cercle(int x, int y) // constructeur paramétrique
    {
        this.x = x ;
        this.y = y ;
    }
}
```

Une même classe peut avoir plusieurs constructeurs qui doivent alors avoir des paramètres différents.

Attention

Si une classe ne contient aucun constructeur, le compilateur ajoutera automatiquement un constructeur sans argument.

Etapes de construction d'un objet

Avant l'appel au constructeur, le programme initialise de manière implicite tous les champs de l'instance en exécutant les initialiseurs de variables. Ces initialiseurs sont **toujours** exécutés en premier lors de l'instanciation. Ainsi les champs d'instances sont initialisés avant l'exécution du constructeur. Il n'est donc pas possible d'avoir un objet dont un champ n'aurait pas été initialisé. Voici les valeurs par défaut :

- Numérique : `0`
- Booléen : `false`
- Référence : `null` (la référence ne désigne aucun objet)

Voici l'ordre des opérations effectuées par l'opérateur `new` :

1. Réservation d'une zone physique en mémoire pour stocker le nouvel objet,
2. Initialisation des champs d'instance,
3. Appel d'un constructeur : constructeur sans argument ou constructeur paramétrique,
4. Retour d'une référence vers l'objet construit.

Chaînage des constructeurs³

Lorsqu'une classe contient plusieurs constructeurs, le contenu de ces derniers est souvent très proche. Par exemple dans le cas de la classe `Cercle`, les 2 constructeurs sont identiques à l'exceptions des valeurs à droite des signes égaux. Pour éviter ce genre de duplication de code (copier/coller de portion de code), il faut chaîner les constructeurs, c'est à dire appeler un constructeur à partir d'un autre constructeur.

En C#, le chaînage se fait lors de la déclaration du constructeur avec le mot clé `this` :

Syntaxe

```
public IdentificateurDeClasse(TypeParametre1 identificateurParametre1, TypeParametre2
identificateurParametre2, ...) : this(parametre1, parametre2, ...)
{...}
```

Pour la classe `Cercle`, cela donne:

```
class Cercle
{
    public int x;
    public int y;

    public Cercle() : this(0, 0) // Le constructeur sans argument est chaîné avec le
constructeur paramétrique
    { } // tout est fait par l'autre constructeur

    public Cercle(int x, int y) // constructeur paramétrique
    {
        this.x = x ;
        this.y = y ;
    }
}
```

Note

Eviter la duplication de code est un principe important de programmation qui facilite la maintenance et permet d'éviter des erreurs. On parle de [DRY principle](#) pour *Don't Repeat Yourself* en anglais.

Exercices³

Exercice 5 : Constructeurs

- - Une classe peut avoir plusieurs constructeurs.

- -

Une classe a toujours un constructeur.

- -

Un constructeur peut avoir un type de retour s'il s'agit d'une référence.

- -

L'opérateur `return` ne peut pas être utilisé dans un constructeur.

- -

Si le développeur ne déclare pas de constructeur sans argument alors le compilateur en ajoute un automatiquement.

- -

Lors de l'appel à l'opérateur `new`, le constructeur sans argument est toujours appelé.

- -

Lors de la création d'un objet, les champs d'instance sont initialisés avant la 1ère instruction du constructeur.

- -

A la déclaration d'un constructeur, celui-ci est implicitement chaîné avec le constructeur sans argument.

- -

Je peux réinitialiser un objet grâce au constructeur.

- -

La référence vers l'objet courant `this` n'existe pas dans un constructeur.

- -

Un constructeur doit avoir le même nom que la classe où il est déclaré.

- -

On doit initialiser la valeur des champs dans un constructeur.

[Corriger](#)

Exercice 6 : Constructeurs

Soit la classe:

```
class A{  
    public int b;  
    public string a;  
    public A(int b){  
        this.b = b;  
    }  
}
```

Quelles instructions sont correctes :

- -

```
A a;
```

- -

```
A a = new A();
```

- -

```
A a = new A(2);
```

- -

```
A a = new A(2,"toto");
```

[Corriger](#)

Exercice 7 : Constructeur

Soit la classe `A`:

```
class A{  
    public int i;  
    public bool b;  
    public A r;  
}
```

Combien la classe `A` possède-t-elle de constructeurs ?

Soit l'objet créé par l'instruction : `new A()`, donnez la valeur de ces 3 attributs:

-
-
-

[Corriger](#)

Exercice 8 : Constructeurs chaînés

Soit la classe `B`:

```
class B{
    int i;
    double d;
    bool b;

    public B(int i, double d) {
        this.b = (b == false);
        this.i = i;
        this.d = d;
    }

    public B(int i) : this(i, i*2.0)
    {}
}
```

Combien la classe `B` possède-t-elle de constructeurs ?

Soit l'objet créé par l'instruction : `new B(1)`, donnez la valeur de ces 3 attributs:

-
-
-

[Corriger](#)

Encapsulation

L'encapsulation consiste à *cacher* le fonctionnement interne d'un composant pour ne laisser apparaître que les éléments utiles à l'utilisateur.

Pour reprendre l'exemple de la voiture, un conducteur est intéressé par certaines fonctions comme *démarrer la voiture, accélérer, freiner...* Le fait de savoir qu'en appuyant sur la pédale de frein, cela va directement comprimer le liquide du circuit de freinage ou bien

enclencher un compresseur auxiliaire n'a pas d'importance de son point de vue. En fait il est même probablement souhaitable que l'utilisateur ne puisse pas interférer avec ces éléments de mécanique interne. L'utilisateur se voit donc présenter une méthode simple pour freiner (appuyer sur la pédale de frein) et les détails de fonctionnement du mécanisme de freinage sont cachés/encapsulés sous le capot.

En programmation, l'encapsulation va consister à choisir les éléments que l'on laisse apparent pour l'utilisateur, on parle de **l'interface**, et ceux que l'on cache. Les principaux avantages de cette approche sont :

- L'utilisabilité : on présente à l'utilisateur une interface claire et épurée qui ne contient que des fonctions utiles à son niveau.
- La souplesse : les éléments de fonctionnement interne étant cachés, on peut les faire évoluer sans toucher à l'interface présentée à l'utilisateur.
- La fiabilité : en empêchant l'utilisateur d'interférer avec le fonctionnement interne du programme, on l'empêche également de mal l'utiliser.
- La clarté du code : en se forçant à identifier les éléments de l'interface et les éléments de fonctionnement interne, on arrive généralement à un code mieux structuré et de meilleure qualité.

Visibilité

En programmation orientée objet, le principal mécanisme de l'encapsulation repose sur les **qualificatifs de visibilité**. Ces qualificatifs permettent de contrôler si une méthode ou un champ peut être utilisé par une autre classe. Nous avons déjà vu le qualificatif **public** qui signifie qu'un membre est accessible sans restriction à tout le monde (il fait parti de l'interface de l'objet). L'opposé du qualificatif public est le qualificatif **private** qui signifie qu'un membre n'est accessible que depuis la classe courante, donc caché au monde extérieur. Essayer d'accéder à un membre privé depuis une autre classe génère une erreur de compilation. Par défaut en C#, un membre dont la visibilité n'est pas précisée est privé.

Par exemple:

```
class A{
    public int champPublic = 1;
    private int champPrive = 2;

    public void MethodePublique(){ }
    private void MethodePrivee(){ }

    public void Test()
    {
        A a = new A();
        champPublic++;
        champPrive++; // ok car nous sommes dans la classe A
    }
}
```

```

        a.champPrive++;    // on peut accéder aux membres privés d'un autre objet
de type A
        MethodePublique();
        MethodePrivee();    // ok car nous sommes dans la classe A
        a.MethodePrivee();    // on peut accéder aux membres privés d'un autre objet
de type A
    }
}

class B{
    public void Test()
    {
        A a = new A();
        a.champPublic++;    // ok car champPublic est public
        // a.champPrive++;    ERREUR car nous sommes dans la classe B
        a.MethodePublique();
        // a.MethodePrivee();    ERREUR car nous sommes dans la classe B
    }
}

```

Attention

Le mot clé **private** n'est pas un élément de sécurité: c'est un guide *fort* pour les développeurs mais un utilisateur mal intentionné pourra toujours le contourner !

Exercice 9 : Visibilité

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

- -
Les membres d'une classe dont la visibilité n'est pas précisée sont considérés comme privés.
- -
Les seuls membres privés auxquels je peux accéder sont ceux de l'instance courante **this**.
- -
Une classe **B** peut accéder aux membres privés d'une classe **A** si elles sont déclarées dans le même fichier.
- -
Dans une même classe, je ne peux pas accéder à un membre privé depuis un membre public.

- -

Les règles de visibilité sont vérifiées à la compilation.

Corriger

Les propriétés³

La problématique qui nous intéresse ici concerne l'intégrité des données. Par exemple, dans un programme de gestion d'un magasin en ligne, on a une classe `Article` avec un champ `prix`. Comment peut-on faire pour assurer que le prix d'un article est toujours strictement positif ? Si on déclare le champ `prix` public alors une autre personne pourrait, par inadvertance ou par malhonnêteté, assigner un prix négatif à un objet.

La solution adoptée dans certains langage (Java/C++) est de déclarer le champ privé avec le mot clé `private` et de déclarer des méthodes dites **accesseurs** :

- **getter** pour récupérer la valeur du champ, et
- **setter** pour modifier sa valeur.

```
class Article{
    private double prix=10;           // champ privé

    public void SetPrix(double nouveauPrix){ // Setter
        if(nouveauPrix>0) // contrôle de validité
            prix = nouveauPrix;
    }

    public double GetPrix(){           // Getter
        return prix; // pas de contrôle ici
    }
}

...

Article a = new Article();
a.SetPrix(20);
WriteLine("Prix de l'article : " + a.GetPrix());
a.SetPrix(-10);
// Passage par Le setter obligatoire
// Passage par Le getter obligatoire
// Sans effet grâce au contrôle effectué par Le setter
```

L'approche par accesseurs comportent des inconvénients:

- cela alourdit le code,
- la maintenance est compliquée car il est difficile de transformer un champ public en un champ privé avec accesseurs dans un programme (le champ public fait parti de l'interface du programme: toute modification sur l'interface peut *casser* les programmes qui utilise cette classe),
- le nommage des accesseurs n'est pas parfaitement standardisé.

La solution proposée en C# sont les **propriétés**. Une propriété est une nouvelle catégorie de membre qui s'utilise comme un champ mais génère automatiquement des appels vers des fonctions **accesseurs**.

Avec une propriété, l'exemple de la classe `Article` s'écrit :

```
class Article{
    double prix ; // champ privé qui contient le prix de base

    // début de la définition de la propriété
    public double Prix {
        get { // définition du getter
            return prix;
        }
        set { // définition du setter
            if(value>0) // value est un mot clé qui désigne le paramètre du
setter
                prix = value;
        }
    }
    // fin de la définition de la propriété
}

...

Article a = new Article();
a.Prix = 20; // génère un appel à set avec value=20
WriteLine("Prix de l'article : " + a.Prix); // génère un appel à get
```

La syntaxe générale pour déclarer une propriété est la suivante:

```
Syntaxe
Visibilite TypeDeDonnees IdentificateurDePropriete {
    get {
        ...
        return valeurDeLaPropriete;
    }
    set {
        ... = value;
    }
}
```

Notez que l'on peut déclarer une propriété sans getter ou sans setter.

Par convention, on donnera toujours le même nom à la propriété et au champ privé qu'elle cache, en faisant commencer le nom du champ par une minuscule et celui de la propriété par une majuscule.

Les propriétés allègent le code et résolvent les problèmes de nommage, mais elles opacifient le code avec des appels de méthodes implicites.

En générale les propriétés ont une visibilité publique, leur but étant de fournir une interface vers l'extérieur de la classe. Néanmoins, on peut modifier individuellement la visibilité des

accesseurs. Par exemple, pour avoir une propriété en lecture/écriture pour la classe mais en lecture seule pour les autres classes :

```
private double prix ;
public double Prix {
    get {...} // getter public
    private set {...} // setter privé
}
```

Dans le cas ou une propriété n'a rien d'autre à faire que de retourner ou affecter directement la valeur d'un champ (sans modification, ni contrôle), on peut utiliser une forme simplifiée dite **propriété auto-implémentée**.

Ainsi la définition:

```
public double Prix { get ; private set ; } // propriété auto-implémentée avec modificateur privé
```

est équivalente à (à la différence qu'aucun champ `prix` n'est accessible même en privé):

```
private double prix ;
public double Prix {
    get { return prix ; }
    private set { prix = value ; }
}
```

Note

Il n'est pas toujours évident de choisir le bon type de membre pour ajouter une nouvelle fonctionnalité dans une classe : attribut ? méthode ? propriété ? constructeur ?...

Les [recommandation de conception](#) du guide de programmation C# peuvent vous aider à choisir l'approche la plus appropriée.

Exercice 10 : Propriétés

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

- - Une propriété peut être de visibilité `public`.
- - Une propriété peut être de visibilité `private`.
- - Une propriété est toujours associée à un champ.

- -
Une propriété doit posséder un `get`.
- -
Une propriété doit posséder un `set`.
- -
Dans le setter d'une propriété, le mot clef `value` désigne la nouvelle valeur à affecter.
- -
Dans le getter d'une propriété, il n'est pas obligatoire de retourner une valeur.
- -
Dans le getter d'une propriété, je n'ai pas le droit de modifier les valeurs des champs de la classe.

Corriger

Exercice 11 : Utilisation des propriétés

Une classe `A` contient le membre suivant : `public string Name {get; set;}`.

Pour chacune de ces instructions, qu'on suppose écrite dans une méthode de la classe `A`, indiquez si elle est correcte ou non:

- -
`Name.set("foo");`
- -
`Name.Set = "foo";`
- -
`Name.set = "foo";`

- -
`Name = "foo";`
- -
`SetName("foo");`
- -
`set("foo");`

[Corriger](#)

Exercice 12 : Effet de bord

Une classe possède une propriété `P`, si je lis 2 fois de suite la valeur de cette propriété, j'obtiendrai :

- -
deux fois la même valeur
- -
deux valeurs différentes
- -
on ne peut pas savoir

[Corriger](#)

Membres statiques

Le qualificateur `static` permet de déclarer des **membres de classe**, aussi appelés **membres statiques**, qui ne dépendront pas d'une instance particulière de la classe. Par exemple la fonction racine carrée, la constante, la fonction `Main` (première fonction exécutée du programme) ne dépendent pas d'un objet particulier; elles existent même si aucun objet n'est créé.

```
class A{
    public static int x;           // champ statique
    public static void Bar(){};    // méthode statique
    public static double UnNombre {get; set;} // propriété statique
}
```

Les champs de classe sont initialisés avant la première utilisation de la classe où ils sont définis.

Les membres de classes peuvent être appelés n'importe quand et n'importe où dans votre programme même si aucun objet du type concerné n'a été créé. Pour accéder à un champ statique ou à une méthode statique, on utilise l'opérateur d'accès `.` préfixé par le nom de la classe (contrairement aux membres d'instance que l'on préfixe par la référence sur un objet). Lors de l'appel à une méthode statique il n'y a donc pas de référence vers une instance courante : le mot clef `this` n'est pas défini. Il n'est donc pas possible d'accéder aux membres d'instances.

Un exemple classique d'utilisation d'un champ statique est la définition d'un compteur d'instances créées pour un classe:

```
class A
{
    double valeur;
    public static int compteur = 0; // nombre d'objets de type A créés

    public A(double valeur){
        this.valeur = valeur;
        A.compteur++; // on vient de créer un nouvel objet
    }

    public static void Test() { // une méthode statique
        A a1 = new A(3.1);
        WriteLine(A.compteur); // 1
        A a2 = new A(3.2);
        WriteLine(A.compteur); // 2
        WriteLine(a1.valeur); // 3.1 accès à un champ d'instance à partir
d'une référence
//WriteLine(this.valeur); ERREUR la référence this n'est pas défini
dans une méthode statique
//WriteLine(a1.compteur); ERREUR accès à partir une référence à un
champ statique
    }
}
```

A la fin de l'exécution de la méthode `Test`, on peut représenter la mémoire de la façon suivante :

Afin de simplifier l'écriture, on peut accéder aux membres statiques de la classe courante sans utiliser le nom de la classe et l'opérateur `.`. Autrement dit, si on ne précise pas de nom de classe avec l'opérateur `.` devant un nom d'identificateur qui correspond à un membre

statique, le compilateur ajoutera automatiquement `NomDeLaClasse.` devant l'identificateur. Le code suivant est donc valide:

```
class A
{
    public static int valeur;

    public void Test()
    {
        valeur = 3; // équivalent à A.valeur = 3
    }
}
```

Note

Le mot **statique** est associé à tout ce qui est calculé au moment de la compilation. Son opposé est le mot **dynamique** qui désigne ce qui est calculé pendant l'exécution du programme.

Exercice 13 : Human VM

```
class A{
    int i = 3;
    static int si = 2;

    void M(){
        i *= 2;
    }

    void static SM(){
        si *= 2;
    }

    void Test()    {
        M();
        SM();
        WriteLine(i);
        WriteLine(si);
    }

    static void Main(string [] args)    {
        A a1 = new A();
        A a2 = new A();
        a1.Test();
        a2.Test();
    }
}
```

Après exécution de la méthode `Main`, ce programme a affiché les 4 lignes de texte suivantes :

- 1.
- 2.
- 3.
- 4.

[Corriger](#)

Exercice 14 : Membres statiques

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

.....

Une classe peut contenir plusieurs membres statiques.

.....

On peut accéder à un membre statique d'une classe sans avoir créé d'objet de cette classe.

.....

Une méthode statique peut accéder à un champ d'instance à travers la référence de l'objet courant `this`.

.....

Une méthode d'instance peut accéder à un champ statique à travers la référence de l'objet courant `this`.

.....

On peut initialiser un champ statique en utilisant les valeurs des champs d'instances.

.....

On peut initialiser un champ d'instance en utilisant les valeurs des champs statiques.

[Corriger](#)

Exercice 15 : Catégories de variables

```
class B{
    static double a;
    int b;
    void F(){
        float c;
    }
}
```

Pour chacune des variables `a`, `b` et `c` dites si il s'agit d'une variable d'instance et/ou de classe et/ou locale.

Variable	Instance	Classe	Locale
<code>a</code>	-	-	-
<code>b</code>	-	-	-
<code>c</code>	-	-	-

[Corriger](#)

Exercice 16 : Statique et règles d'accès aux champs

```
class A{
    public static int a;
    public int b;
}

class B{
    public static void Main(){
        A.a++;
        A.b++;
        A obj= new A();
        obj.a++;
        obj.b++;
    }
}
```

- -
La ligne 8 est correcte.
- -
La ligne 9 est correcte.
- -
La ligne 11 est correcte.
- -
La ligne 12 est correcte.

[Corriger](#)

Exercice 17 : Statique et règles d'accès aux champs

```

class A{
    static int a;
    int b;

    public void F1(){
        a++;
        b++;
    }

    public static void F2(){
        a++;
        b++;
    }
}

```

- -
La ligne 6 est correcte.
- -
La ligne 7 est correcte.
- -
La ligne 11 est correcte.
- -
La ligne 12 est correcte.

[Corriger](#)

Exercices [↗](#)

Exercice 18 : Human VM

Soit la classe `C` et la fonction `Test`

```

class C
{
    public int a, b;
    public C bb;
    public C(int _a, int _b) { a=_a; b=_b; }
}

...

Test()

```

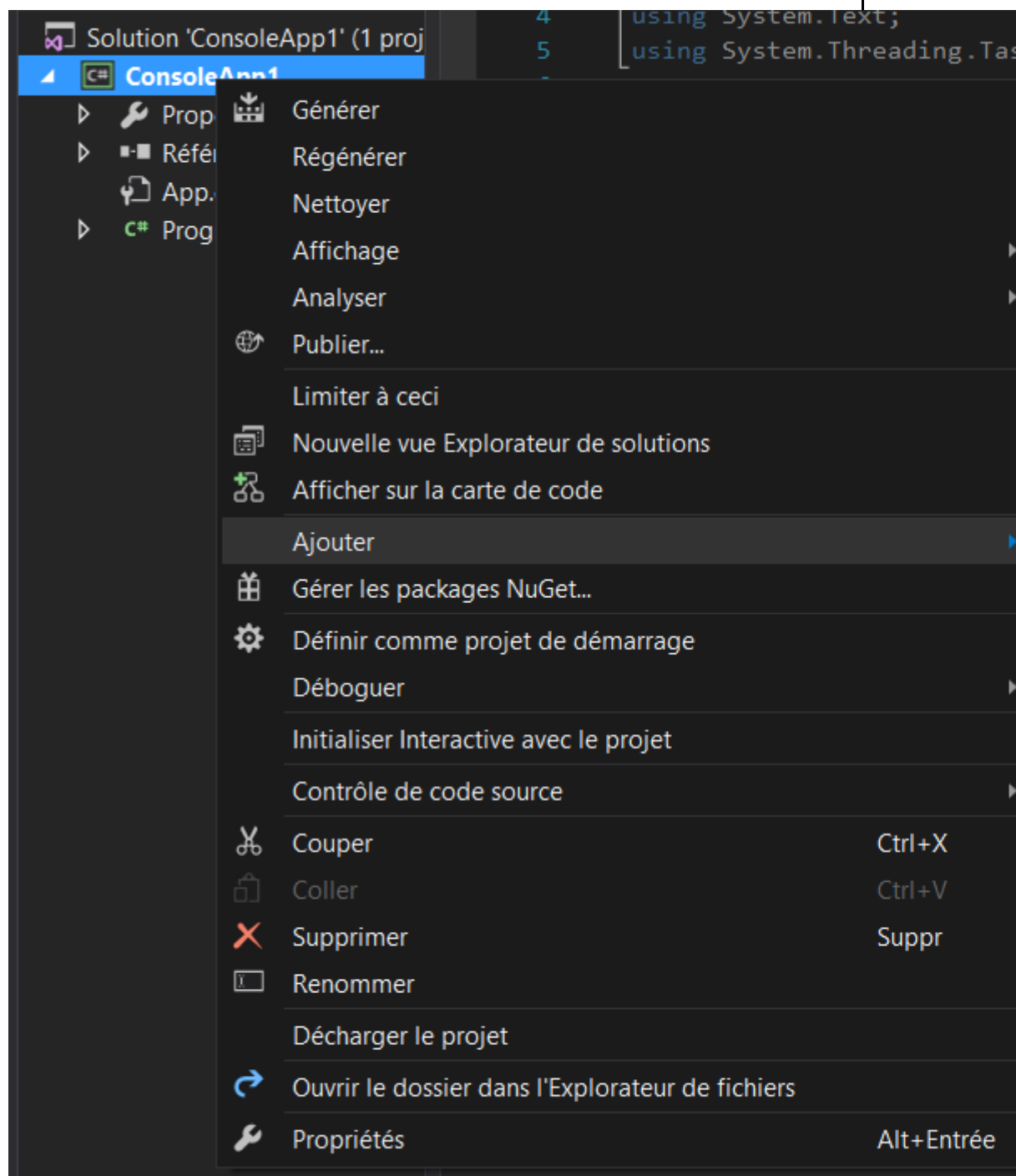
```
{  
    C[] T = new C[5];  
    for (int i = 0 ; i < 5 ; i++)  
    {  
        T[i] = new C(i,i);  
        i++;  
    }  
}
```

Dessinez l'état de la mémoire à la fin de la fonction `Test` (un papier et un crayon reste la façon la plus simple de dessiner !)

Exercice 19 : Création d'une classe

Sous Visual Studio, créez un nouveau projet de type Console:

1. Créez une classe `Rectangle`: cliquez droit sur le nom du projet dans l'explorateur de solution, menu *Ajouter -> Classe*, renseignez le nom de la classe et cliquez sur OK.



2. Ajoutez des attributs à la classe : sa largeur `largeur`, sa hauteur `hauteur` en pixel et sa position à l'écran, `positionX` et `positionY`, en pixel.
3. Ajoutez un constructeur paramétrique qui initialise la taille et la position de l'objet.
4. Ajoutez une méthode d'instance `Aire` (sans argument) qui retourne l'aire du rectangle.
5. Dans la méthode `Main` de la classe `Program` :

- A. Créez une nouvelle instance de la classe `Rectangle` de dimension 20 par 30 à la position (15,20).
 - B. Affichez les valeurs des attributs de l'objet créé.
 - C. Affichez l'aire de l'objet créé (utilisez la méthode `Aire`) afin de la calculer.
 - D. Exécutez votre programme et vérifiez que tout fonctionne correctement (ajoutez l'instruction `Console.Read();` à la fin du `Main` pour éviter que la fenêtre ne se ferme tout de suite).
6. Ajoutez un constructeur sans argument qui initialise la forme comme un carré de côté 10 et de position (0,0). Respectez le principe DRY : utilisez le chainage de constructeurs pour que le corps de votre constructeur ne contienne aucune instruction !
7. Ajoutez une méthode `DoublerTaille` qui multiplie la largeur et la longueur du rectangle par 2.
8. Dans la méthode `Main` de la classe `Program` :
 - A. Créez une nouvelle instance de la classe `Rectangle` avec votre constructeur sans argument.
 - B. Doublez la taille du rectangle créé avec la méthode `DoublerTaille`.
 - C. Affichez les dimensions du rectangle pour vérifier que tout a bien fonctionné.
9. Ajoutez une méthode `TestePlusPetit` qui prend en paramètre un rectangle `r` et retourne vrai si l'aire de `r` est plus petite que l'aire de l'instance courante.
10. Dans la méthode `Main` de la classe `Program` :
 - A. Testez la méthode `TestePlusPetit` avec les 2 références de `Rectangle` déjà créées.
11. Créez le constructeur de copie : constructeur qui reçoit en paramètre une référence sur un objet du même type et qui initialise à l'identique l'objet en cours de création. Respectez le principe DRY : utilisez le chainage de constructeurs pour que le corps de votre constructeur ne contienne aucune instruction !
12. Dans la méthode `Main` de la classe `Program` :
 - A. Créez une copie du premier rectangle créé.
 - B. Modifiez la largeur de la copie.

C. Vérifiez que la largeur du premier rectangle n'a pas changé.

Exercice 20 : Human VM

Soit la classe `C` et la fonction `Test`

```
class C
{
    public int a,b;
    public C(int _a, int _b) { a=_a; b=_b; }
}

...

Test()
{
    C[] T = new C[5];
    for (int i = 0 ; i < 5 ; i++)
    {
        T[i] = new C(i,i+1);
    }
    T[2] = T[4];
    for (int i = 0 ; i < 5 ; i++)
    {
        T[i].a += T[i].b;
    }
}
```

Dessinez l'état de la mémoire à la fin de la fonction `Test` (un papier et un crayon reste la façon la plus simple de dessiner !)

Exercice 21 : Copie profonde

Une pratique courante est de proposer dans chaque classe un constructeur par copie, c'est-à-dire un constructeur qui prend en paramètre un objet du même type que celui à construire et qui construit une **copie profonde (deep copy)** de cet objet. Pour réaliser une copie profonde d'objet, il faut non seulement dupliquer l'objet mais également réaliser une copie profonde de tous les objets référencés par cet objet : l'objet d'origine et l'objet copié sont complètement indépendants. On parle de clonage.

Téléchargez et ouvrez cette [solution Visual Studio](#).

1. Ajoutez à la classe `Point` un constructeur prenant en paramètre un objet de type `Point` et qui construit une copie profonde de ce point.
2. Ajoutez à la classe `Polygone` un constructeur prenant en paramètre un objet de type `Polygone` et qui construit une copie profonde de ce polygone.
3. Décommentez le contenu du `Main` et exécutez le programme pour vérifier que tout fonctionne.

Donnez l'état de la mémoire à la fin de la fonction suivante (un papier et un crayon reste la façon la plus simple de dessiner !):

```
Point p1 = new Point(1,1);
Point p2 = p1;
Point p3 = new Point(p1);
Point [] pts = {p1, p2, p3};
Polygone po1 = new Polygone(pts);
Polygone po2 = new Polygone(pts);
Polygone po3 = new Polygone(po1);
p1.x=2;
p2.x=3;
p3.x=4;
po1.points[0].y=5;
po2.points[1].y=6;
po3.points[2].y=7;
```

Exercice 22 : Propriétés

Dans un nouveau projet Console sous Visual Studio, écrivez une classe `Duree` qui devra stocker une durée avec une précision à la seconde. La classe devra posséder 2 propriétés `DureeSeconde` et `DureeMinute` qui permettent de lire et modifier la durée avec des valeurs exprimées respectivement en secondes et en minutes. Une durée ne sera jamais négative. Pour exprimer une durée en minutes, on arrondira toujours à la valeur supérieure.

Exemple d'utilisation:

```
Duree d = new Duree(70); // durée de 70 secondes
WriteLine(d.DureeSeconde); // 70
d.DureeMinute = -2;
WriteLine(d.DureeSeconde); // 70
WriteLine(d.DureeMinute); // 2
d.DureeMinute = 2;
WriteLine(d.DureeSeconde); // 120
```

Exercice 23 : Course de lapin

Dans un nouveau projet Console sous Visual Studio:

1. Créez une classe `Lapin` qui contient les propriétés suivantes: `Surnom` (`String`), `Age` (`int`), `Position` (`int`). Vous implémenterez complètement les propriétés `Surnom` et `Age` et vous utiliserez une propriété auto-implémentée pour `Position`.
2. Ecrivez un constructeur permettant d'initialiser les propriétés `Surnom` et `Age` (`Position` sera toujours initialisée à 0).
3. Ajoutez un champ statique `nombreDeLapins` qui compte le nombre de lapins créés. Modifiez le constructeur pour que ce champ soit automatiquement incrémenté lors de la création d'un nouveau lapin.

4. Ajoutez une méthode statique `NombreDeLapins()` qui retourne le nombre lapins créé depuis le lancement du programme.
5. Ajoutez une méthode `void Avancer()` à la classe `Lapin`. Cette méthode ajoute une valeur aléatoire comprise entre 0 et 5 à la position du lapin. La méthode d'instance `Next` de la classe `Random` génère un nombre (pseudo) aléatoire.
6. Dans la méthode `Main` de la classe `Program`, créez trois lapins avec des noms et des ages différents.
7. Créez un tableau de lapins et rangez-y les trois lapins créés précédemment.
8. Appelez la méthode `Avancer()` 100 fois sur chacun des 3 lapins.
9. Le lapin qui a gagné est celui qui a la position la plus grande à la fin. Affichez le nom du lapin qui a gagné.

© Droits d'auteur B. Perret, ESIEE Paris 2022.

Compilé avec [Sphinx](#) en utilisant un [thème](#) fourni par [Read the Docs](#).

P00 et C#

- [Introduction à l'environnement .net](#)
- [Base du langage C#](#)

- **[Les classes](#)**

- [Une extension de la notion de type](#)
- [Champs et méthodes d'instance](#)
- [Création d'un objet](#)
- [Opérateur d'accès](#)
- [Les constructeurs](#)
- [Encapsulation](#)
- [Membres statiques](#)
- [Exercices](#)

- [L'héritage](#)
- [Compléments de langage](#)
- [Fonctions avancées](#)
- [Projet - Space Invaders](#)
- [Git](#)
- [Space Invaders - Piste Verte](#)

-

- Les classes
-

Les classes

Une extension de la notion de type

Un **type de données**, ou simplement **type**, désigne la conjugaison de deux aspects fondamentaux de la programmation :

1. un regroupement de données, on parle **d'agrégation**, et,
2. des méthodes pour initialiser ces données et les manipuler.

Par exemple le type `int` décrit comment stocker un nombre entier signé sur 32 bits (les données) ainsi que les opérateurs communs pour manipuler ces nombres `+`, `*`, `-` ... (les méthodes). Notez que le type `int`, n'existe pas en mémoire, c'est la variable de type `int` qui va déclencher la réservation d'une case mémoire pour son stockage. Le type `int` est unique mais permet la création de plusieurs variables de type `int`.

En dehors des types de base comme les types numériques, il vous est possible de créer ou d'utiliser d'autres types de données à l'intérieur de vos programmes. Ces types plus complexes seront désignés sous le terme de **classes**. Dit autrement, une classe est une description d'un nouveau type de données. Tout comme pour le type `int`, une classe est unique mais permet de **créer** (ou **instancier**), plusieurs **objets** (ou **instances**).

Attention

La classe et l'objet sont des concepts liés mais intrinsèquement différents. Par exemple, prenons un objet réel comme une voiture. Le modèle numérique de la voiture décrit sa forme, ses fonctions, comment elle doit être fabriquée. Ce modèle n'est pas une voiture. De la même manière une classe n'est pas un objet.



Il y a 1 type pour 0 à n objets créés. A partir d'un plan, plusieurs pièces peuvent être créées, ou aucune aussi. Les pièces ainsi créées partagent les informations décrites dans le plan. Les pièces existent indépendamment. Une fois la pièce A intégrée dans une voiture, elle n'est plus en relation avec la pièce B assemblée dans un avion. Ces deux pièces sont identiques au sens où elles sont deux pièces du même modèle. Cependant elles sont différentes au sens où elles vont vivre indépendamment. Ainsi la pièce B pourra être remplacée car cassée au bout d'un an alors que l'autre pourra vivre dix ans.

Les objets peuvent avoir des attributs propres, dans ce cas, ses attributs doivent être décrits dans le plan. Par exemple, un autoradio contient une liste de stations préférées configurées par l'utilisateur. Le modèle de l'autoradio indique combien de canaux préférés peuvent être choisis cependant aucun n'est établi avant que l'utilisateur ne les choisissent. De même, une voiture C3 est éditée suivant plusieurs couleurs de carrosserie et plusieurs types de moteurs. Pourtant une C3 noire (on parle ici d'un objet) et une C3 blanche (on parle toujours d'un objet) reste des voitures du modèle C3. Il en va de même pour une C3 diesel et une C3 essence.

Note

Nous dirons parfois *un objet cercle* au lieu d'*un objet de type Cercle*.

La déclaration d'une nouvelle classe se fait grâce au mot clef **class** suivi d'un identificateur : le nom de la nouvelle classe. Le corps de la classe (son contenu) est placé entre accolades. En général, une classe *A* est déclarée directement dans un fichier portant le même nom *A.cs* (une classe par fichier). Par convention, les identificateur de classe utiliseront également la norme CamelCase avec la premier lettre en majuscule.

Syntaxe

```
class Identificateur{  
    // corps de la classe  
}
```

Exercice 1 : Instanciation

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse :

- -
On ne peut instancier qu'un objet à partir d'une classe.
- -
Une classe décrit un type de données.
- -
Une classe permet de créer plusieurs objets de même caractéristiques.
- -
Une classe peut porter plusieurs noms différents.

[Corriger](#)

Note

Les recommandations de nommage pour les classes sont traitées [ici](#) .

Champs et méthodes d'instance

Une variable propre à un objet (la couleur, l'immatriculation de la voiture) est qualifiée de **champ d'instance** (ou **variable d'instance**, ou **attribut** ou **instance field**). Entre deux objets de type *Voiture*, ces paramètres peuvent avoir des valeurs différentes. Une fonction propre à un objet est qualifiée de **méthode d'instance** (**method**). Cette méthode doit être appelée depuis une instance particulière et n'influe que sur le comportement de cet objet : les différentes instances d'une même classe sont indépendantes.

Les champs et les méthodes d'instances d'une classe sont appelés les **membres d'instance** (**instance members**) de la classe. Chaque membre d'instance est précédé d'un **qualificatif de visibilité** qui donne les droits d'accès depuis *l'extérieur*, c'est-à-dire

depuis du code ne se trouvant pas la classe courante (celle que l'on est en train d'écrire). Le qualificatif **public** ouvre l'accès sans restriction. Pour le moment, on utilisera le qualificatif public mais nous reviendrons là-dessus dans le chapitre **Encapsulation**.

La déclaration de champs et de méthodes d'instance se fait à l'intérieur de la classe:

Syntaxe

```
class Identificateur{

    // déclaration des champs d'instances
    public TypeChamp1 identificateurChamp1;
    public TypeChamp2 identificateurChamp2;
    ...

    // déclaration des méthodes d'instances
    public TypeDeRetour1 IdentificateurMethode1(TypeParametre11
identificateurParametre11, ...) {...}
    public TypeDeRetour2 IdentificateurMethode2(TypeParametre21
identificateurParametre21, ...) {...}
    ...
}
```

Exemple:

```
class Voiture{

    public int couleur ;
    public int puissance ;

    public void Demarrer() { ... }
    public void Arrêter() { ... }
}
```

L'ordre de déclaration des membres d'une classe n'a pas d'importance. Par convention, on déclarera les champs d'instance en premier, puis les méthodes.

Exercice 2 : Membres d'instance

- -
Tous les objets d'une même classe possèdent les mêmes champs et les mêmes méthodes d'instance.
- -
Tous les champs des objets d'une même classe possèdent des valeurs identiques.
- -

Lorsque je déclare un champ d'instance, le CLR va réserver de la mémoire pour ce champs au lancement du programme.

- -
Variable d'instance est un synonyme de champ d'instance.
- -
Les champs et les méthodes d'instance font partis des membres d'instance d'une classe.
- -
Une classe doit contenir des champs d'instance.
- -
Une classe doit contenir des méthodes d'instance.

Corriger

Afin de facilement identifier le rôle d'un membre d'instance, on pourra s'en tenir aux [conventions de nommage](#) données dans le guide de programmation C#.

Création d'un objet³

La **création** d'un objet, ou **instanciation**, est réalisé avec l'opérateur **new** qui retourne une **référence** vers le nouvel objet créé.

Syntaxe

```
identificateurVariable = new IdentificateurClasse(parametre1, parametre2, ...);
```

Il y a deux actions effectuées :

1. Création d'un objet de type *IdentificateurClasse* en utilisant les paramètres éventuels : c'est la partie à droite du signe égale;
2. Initialisation de la variable *identificateurVariable* avec la référence retournée par l'opérateur `new`.

Exemple:

```

/* déclaration d'une variable de type Bitmap
   et initialisation avec la référence vers un nouvel objet de type Bitmap avec les
   paramètre (400, 200)*/
Bitmap image = new Bitmap(400,200);
/* déclaration d'une variable de type Voiture
   et initialisation avec une référence vers un nouvel objet de type Voiture */
Voiture voiture = new Voiture();

```

Attention

Les classes font parties de la catégorie **type référence**. Cela signifie qu'une variable dont le type correspond à une classe ne contient pas directement un objet mais une référence vers un objet. Ce lien de *référence* est généralement représenté par une flèche et on dit que la variable *référence* l'objet ou *pointe vers* l'objet (en référence à la notion de pointeur en C/C++).

Par abus de langage, une variable de catégorie **type référence** sera simplement appelée **référence**. Ainsi, dans l'exemple précédent que `image` est une référence de type `Bitmap`.

Considérons le code :

```

class C{
    public int i;
    public bool b;
}

...

int v = 2;
C c = new C();

```

La mémoire contient 2 variables, `v` et `c`, et un objet de type `C`. Elle peut être représentée ainsi:

Représentation schématique de l'état de la mémoire du programme. [↗](#)

En pratique, la mémoire de l'ordinateur est divisée en octets. Chaque octet reçoit un numéro d'identification unique allant de 0 jusqu'à la taille maximale physiquement disponible (la mémoire peut être vue comme un tableau d'octets). Ce numéro est l'adresse en mémoire. L'adresse identifie ainsi un endroit unique dans la mémoire. Pour stocker un objet, le système réserve un bloc de mémoire. Ainsi, un objet de 10 octets peut occuper les cellules contigües 80, 81, 82... 89. La référence va contenir l'adresse de la première cellule soit la valeur 80. Durant les exercices, plutôt que de créer des adresses fictives, il est plus simple de schématiser la référence comme une flèche vers une autre zone de la mémoire.

Exercice 3 : Référence et création d'instances

- -
Si je crée trois références alors elles désignent trois objets différents.
- -
Il peut exister plusieurs variables qui référencent un même objet.
- -
Il existe trois catégories de variables : valeur, référence et objet.
- -
La déclaration de variable `A v;` ou `A` est un identificateur de classe ne provoque pas la création d'un objet de type `A`.
- -
Pour créer un nouvel objet à partir d'un objet existant, je déclare une variable que j'initialise avec une référence vers cet objet.
- -
L'instruction `Voiture v = v;` est-elle valide ?
- -
Les instructions `Voiture v = new Voiture(); v=v;` sont-elles valides ?

[Corriger](#)

Opérateur d'accès ³

L'**accès aux membres d'instances**, champs et méthodes, se fait à travers l'utilisation d'une référence sur l'objet concerné et de l'**opérateur d'accès point (dot)** `.`

Syntaxe

```
// accès à un champ d'instance
reference.identificateurChamps
```

```
// appel d'une methode d'instance
reference.IdentificateurMethode(parametre1, parametre2, ...)
```

Exemple:

```
Voiture referenceVoiture = new Voiture();
referenceVoiture.Demarre();
int p = referenceVoiture.puissance;
```

Une référence qui contient la valeur spéciale `null` ne désigne aucun objet. Tenter d'appliquer l'opérateur d'accès `.` sur une référence `null` est une erreur de programmation qui provoque un crash du programme à l'exécution.

Instance courante

Nous venons de voir que toutes les méthodes d'instance sont appelées à partir d'une référence (à gauche de l'opérateur `.`). Dans la méthode d'instance, cette référence est appelée **instance courante** et est désignée par le mot clé `this`. L'instance courante est une référence sur l'objet sur lequel on est en train de travailler.

Exemple:

```
class Compteur
{
    public int valeur;

    public void SetValeur(int nouvelleValeur)
    {
        this.valeur = nouvelleValeur; // this.valeur désigne le champ valeur de
        // l'objet sur lequel la méthode SetValeur est appelée
    }
}

Compteur compteur = new Compteur();
compteur.SetValeur(10);
```

Un objet de type `Compteur` est créé. Cet objet est référencé par la variable `compteur`. L'opérateur d'accès est appliqué sur la référence `compteur`. Il permet ainsi d'accéder à la fonction `SetValeur()` de l'objet concerné. L'opérateur d'accès fixe l'objet référencé comme instance courante lors de l'exécution de la fonction `SetValeur()` (`this` est égal à `compteur`).

Afin de simplifier l'écriture, on peut accéder aux membres d'instance de l'instance courante sans utiliser la référence `this` et l'opérateur `.`. Si on ne précise pas de référence avec l'opérateur `.` devant un nom d'identificateur qui correspond à un membre d'instance, le compilateur ajoutera automatiquement `this.` devant l'identificateur. Le code suivant est donc équivalent au précédent:

```
class Compteur
```

```

{
    public int valeur;

    public void SetValeur(int nouvelleValeur)
    {
        valeur = nouvelleValeur; // équivalent à this.valeur = ...
    }
}

```

Note

A l'intérieur du code d'une méthode d'instance, l'accès aux membres de l'instance courante peut donc se faire directement sans passer par l'opérateur d'accès. Les normes d'écriture privilégient généralement cette facilité. Une exception notable apparaît lorsqu'il est nécessaire de faire la différence entre un champ d'instance et une variable locale ayant le même identificateur.

```

class Compteur
{
    public int valeur;

    public void SetValeur(int valeur) // Le paramètre à le même nom que le champ
    {
        this.valeur = valeur; // this.valeur désigne le champ d'instance, alors que
        // valeur désigne la variable locale
    }
}

```

Indépendance entre objets [§](#)

Si l'on crée deux instances de la classe `Compteur` :

```

Compteur c1 = new Compteur();
Compteur c2 = new Compteur();

```

Deux zones mémoires sont réservées pour stocker chaque instance. Deux références `c1` et `c2` sont créées et associées à chaque objet. On peut représenter la mémoire de la façon suivante :

Si on exécute l'instruction:

```

c1.SetValeur(10);

```

L'opérateur d'accès `.` appliqué sur la référence `c1` m'indique que l'instance courante `this` va être l'objet référencé par `c1`. Ainsi, c'est le champ `valeur` de cet objet qui va être modifié et changé pour la valeur 10. Le 2ème objet ne subit aucune modification et la mémoire devient:

Exercices

Exercice 4 : Membres d'instances

- -
En modifiant une variable `v` de type référence, je ne modifie pas l'objet référencé par `v`.
- -
Si je change la valeur d'un champ d'instance, l'objet n'est plus considéré comme appartenant à sa classe d'origine.
- -
Une fois l'objet créé et ses variables d'instance initialisées, je ne peux plus les modifier.
- -
Je peux accéder directement à un objet sans passer par une référence.
- -
Si j'appelle une méthode depuis une référence et que cette référence a pour valeur `null` alors le programme plante.
- -
Soient deux références `r1` et `r2` qui désignent le même objet, si je change `r1` pour qu'elle réfère un autre objet, alors `r2` désigne aussi ce nouvel objet.
- -
Si aucune référence ne désigne un objet, alors je ne peux pas accéder aux attributs de cet objet.

- Dans une méthode d'instance, `this` est toujours une référence vers un objet de la classe courante.
- Dans une méthode d'instance, `this.v` et `v` peuvent désigner 2 choses différentes.

Corriger

Les constructeurs

Chaque classe contient un ou plusieurs **constructeurs**. Cette fonction particulière est appelée lors de la création d'un objet par l'opérateur `new`. Le but du constructeur est d'initialiser les champs d'instances de l'objet. Un constructeur est déclaré comme une méthode sans type de retour et dont le nom est identique à celui de la classe :

Syntaxe

```
public IdentificateurDeClasse(TypeParametre1 identificateurParametre1, TypeParametre2  
identificateurParametre2, ...)  
{...}
```

Exemple :

```
class Cercle  
{  
    public int x;  
    public int y;  
  
    public Cercle() // constructeur sans argument  
    {  
        x = 10;  
        y = 10;  
    }  
  
    public Cercle(int x, int y) // constructeur paramétrique  
    {  
        this.x = x ;  
        this.y = y ;  
    }  
}
```

Une même classe peut avoir plusieurs constructeurs qui doivent alors avoir des paramètres différents.

Attention

Si une classe ne contient aucun constructeur, le compilateur ajoutera automatiquement un constructeur sans argument.

Etapes de construction d'un objet

Avant l'appel au constructeur, le programme initialise de manière implicite tous les champs de l'instance en exécutant les initialiseurs de variables. Ces initialiseurs sont **toujours** exécutés en premier lors de l'instanciation. Ainsi les champs d'instances sont initialisés avant l'exécution du constructeur. Il n'est donc pas possible d'avoir un objet dont un champ n'aurait pas été initialisé. Voici les valeurs par défaut :

- Numérique : `0`
- Booléen : `false`
- Référence : `null` (la référence ne désigne aucun objet)

Voici l'ordre des opérations effectuées par l'opérateur `new` :

1. Réservation d'une zone physique en mémoire pour stocker le nouvel objet,
2. Initialisation des champs d'instance,
3. Appel d'un constructeur : constructeur sans argument ou constructeur paramétrique,
4. Retour d'une référence vers l'objet construit.

Châinage des constructeurs

Lorsqu'une classe contient plusieurs constructeurs, le contenu de ces derniers est souvent très proche. Par exemple dans le cas de la classe `Cercle`, les 2 constructeurs sont identiques à l'exceptions des valeurs à droite des signes égaux. Pour éviter ce genre de duplication de code (copier/coller de portion de code), il faut chaîner les constructeurs, c'est à dire appeler un constructeur à partir d'un autre constructeur.

En C#, le chaînage se fait lors de la déclaration du constructeur avec le mot clé `this` :

Syntaxe

```
public IdentificateurDeClasse(TypeParametre1 identificateurParametre1, TypeParametre2
identificateurParametre2, ...) : this(parametre1, parametre2, ...)
{...}
```

Pour la classe `Cercle`, cela donne:

```
class Cercle
{
```

```

    public int x;
    public int y;

    public Cercle() : this(0, 0) // Le constructeur sans argument est chaîné avec Le
    constructeur paramétrique
    { } // tout est fait par l'autre constructeur

    public Cercle(int x, int y) // constructeur paramétrique
    {
        this.x = x ;
        this.y = y ;
    }
}

```

Note

Eviter la duplication de code est un principe important de programmation qui facilite la maintenance et permet d'éviter des erreurs. On parle de [DRY principle](#) pour *Don't Repeat Yourself* en anglais.

Exercices

Exercice 5 : Constructeurs

- -
Une classe peut avoir plusieurs constructeurs.
- -
Une classe a toujours un constructeur.
- -
Un constructeur peut avoir un type de retour s'il s'agit d'une référence.
- -
L'opérateur `return` ne peut pas être utilisé dans un constructeur.
- -
Si le développeur ne déclare pas de constructeur sans argument alors le compilateur en ajoute un automatiquement.
- -

Lors de l'appel à l'opérateur `new`, le constructeur sans argument est toujours appelé.

- -
Lors de la création d'un objet, les champs d'instance sont initialisés avant la 1ère instruction du constructeur.
- -
A la déclaration d'un constructeur, celui-ci est implicitement chaîné avec le constructeur sans argument.
- -
Je peux réinitialiser un objet grâce au constructeur.
- -
La référence vers l'objet courant `this` n'existe pas dans un constructeur.
- -
Un constructeur doit avoir le même nom que la classe où il est déclaré.
- -
On doit initialiser la valeur des champs dans un constructeur.

[Corriger](#)

Exercice 6 : Constructeurs

Soit la classe:

```
class A{  
    public int b;  
    public string a;  
    public A(int b){  
        this.b = b;  
    }  
}
```

Quelles instructions sont correctes :

- -

`A a;`

- -

`A a = new A();`

- -

`A a = new A(2);`

- -

`A a = new A(2, "toto");`

[Corriger](#)

Exercice 7 : Constructeur

Soit la classe `A` :

```
class A{
    public int i;
    public bool b;
    public A r;
}
```

Combien la classe `A` possède-t-elle de constructeurs ?

Soit l'objet créé par l'instruction : `new A()`, donnez la valeur de ces 3 attributs:

- `i`
- `b`
- `r`

[Corriger](#)

Exercice 8 : Constructeurs chaînés

Soit la classe `B` :

```
class B{
    int i;
    double d;
    bool b;

    public B(int i, double d) {
        this.b = (b == false);
    }
}
```

```
        this.i = i;
        this.d = d;
    }

    public B(int i) : this(i, i*2.0)
    {}
}
```

Combien la classe `B` possède-t-elle de constructeurs ?

Soit l'objet créé par l'instruction : `new B(1)`, donnez la valeur de ces 3 attributs:

- `i`
- `d`
- `b`

[Corriger](#)

Encapsulation

L'encapsulation consiste à *cacher* le fonctionnement interne d'un composant pour ne laisser apparaître que les éléments utiles à l'utilisateur.

Pour reprendre l'exemple de la voiture, un conducteur est intéressé par certaines fonctions comme *démarrer la voiture, accélérer, freiner...* Le fait de savoir qu'en appuyant sur la pédale de frein, cela va directement comprimer le liquide du circuit de freinage ou bien enclencher un compresseur auxiliaire n'a pas d'importance de son point de vue. En fait il est même probablement souhaitable que l'utilisateur ne puisse pas interférer avec ces éléments de mécanique interne. L'utilisateur se voit donc présenter une méthode simple pour freiner (appuyer sur la pédale de frein) et les détails de fonctionnement du mécanisme de freinage sont cachés/encapsulés sous le capot.

En programmation, l'encapsulation va consister à choisir les éléments que l'on laisse apparent pour l'utilisateur, on parle de **l'interface**, et ceux que l'on cache. Les principaux avantages de cette approche sont :

- L'utilisabilité : on présente à l'utilisateur une interface claire et épurée qui ne contient que des fonctions utiles à son niveau.
- La souplesse : les éléments de fonctionnement interne étant cachés, on peut les faire évoluer sans toucher à l'interface présentée à l'utilisateur.

- La fiabilité : en empêchant l'utilisateur d'interférer avec le fonctionnement interne du programme, on l'empêche également de mal l'utiliser.
- La clarté du code : en se forçant à identifier les éléments de l'interface et les éléments de fonctionnement interne, on arrive généralement à un code mieux structuré et de meilleure qualité.

Visibilité

En programmation orientée objet, le principal mécanisme de l'encapsulation repose sur les **qualificatifs de visibilité**. Ces qualificatifs permettent de contrôler si une méthode ou un champ peut être utilisé par une autre classe. Nous avons déjà vu le qualificatif **public** qui signifie qu'un membre est accessible sans restriction à tout le monde (il fait parti de l'interface de l'objet). L'opposé du qualificatif public est le qualificatif **private** qui signifie qu'un membre n'est accessible que depuis la classe courante, donc caché au monde extérieur. Essayer d'accéder à un membre privé depuis une autre classe génère une erreur de compilation. Par défaut en C#, un membre dont la visibilité n'est pas précisée est privé.

Par exemple:

```
class A{
    public int champPublic = 1;
    private int champPrive = 2;

    public void MethodePublique(){ }
    private void MethodePrivee(){ }

    public void Test()
    {
        A a = new A();
        champPublic++;
        champPrive++; // ok car nous sommes dans la classe A
        a.champPrive++; // on peut accéder aux membres privés d'un autre objet
de type A
        MethodePublique();
        MethodePrivee(); // ok car nous sommes dans la classe A
        a.MethodePrivee(); // on peut accéder aux membres privés d'un autre objet
de type A
    }
}

class B{
    public void Test()
    {
        A a = new A();
        a.champPublic++; // ok car champPublic est public
        // a.champPrive++; ERREUR car nous sommes dans la classe B
        a.MethodePublique();
        // a.MethodePrivee(); ERREUR car nous sommes dans la classe B
    }
}
```

Attention

Le mot clé **private** n'est pas un élément de sécurité: c'est un guide *fort* pour les développeurs mais un utilisateur mal intentionné pourra toujours le contourner !

Exercice 9 : Visibilité

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

- -
Les membres d'une classe dont la visibilité n'est pas précisée sont considérés comme privés.
- -
Les seuls membres privés auxquels je peux accéder sont ceux de l'instance courante **this**.
- -
Une classe **B** peut accéder aux membres privés d'une classe **A** si elles sont déclarées dans le même fichier.
- -
Dans une même classe, je ne peux pas accéder à un membre privé depuis un membre public.
- -
Les règles de visibilité sont vérifiées à la compilation.

Corriger

Les propriétés³

La problématique qui nous intéresse ici concerne l'intégrité des données. Par exemple, dans un programme de gestion d'un magasin en ligne, on a une classe **Article** avec un champ **prix**. Comment peut-on faire pour assurer que le prix d'un article est toujours strictement positif ? Si on déclare le champ *prix* public alors une autre personne pourrait, par inadvertance ou par malhonnêteté, assigner un prix négatif à un objet.

La solution adoptée dans certains langage (Java/C++) est de déclarer le champ privé avec le mot clé `private` et de déclarer des méthodes dites **accesseurs** :

- **getter** pour récupérer la valeur du champ, et
- **setter** pour modifier sa valeur.

```
class Article{
    private double prix=10;           // champ privé

    public void SetPrix(double nouveauPrix){ // Setter
        if(nouveauPrix>0) // contrôle de validité
            prix = nouveauPrix;
    }

    public double GetPrix(){           // Getter
        return prix; // pas de contrôle ici
    }
}

...

Article a = new Article();
a.SetPrix(20);                        // Passage par Le setter obligatoire
WriteLine("Prix de l'article : " + a.GetPrix()); // Passage par Le getter obligatoire
a.SetPrix(-10);                       // Sans effet grâce au contrôle effectué
// par Le setter
```

L'approche par accesseurs comportent des inconvénients:

- cela alourdit le code,
- la maintenance est compliquée car il est difficile de transformer un champ public en un champ privé avec accesseurs dans un programme (le champ public fait parti de l'interface du programme: toute modification sur l'interface peut *casser* les programmes qui utilise cette classe),
- le nommage des accesseurs n'est pas parfaitement standardisé.

La solution proposée en C# sont les **propriétés**. Une propriété est une nouvelle catégorie de membre qui s'utilise comme un champ mais génère automatiquement des appels vers des fonctions **accesseurs**.

Avec une propriété, l'exemple de la classe `Article` s'écrit :

```
class Article{
    double prix ; // champ privé qui contient Le prix de base

    // début de La définition de La propriété
    public double Prix {
        get { // définition du getter
            return prix;
        }
        set { // définition du setter
            if(value>0) // value est un mot clé qui désigne Le paramètre du
setter
                prix = value;
        }
    }
    // fin de La définition de La propriété
```

```

}

...

Article a = new Article();
a.Prix = 20; // génère un appel à set avec value=20
WriteLine("Prix de l'article : " + a.Prix); // génère un appel à get

```

La syntaxe générale pour déclarer une propriété est la suivante:

```

Syntaxe
Visibilite TypeDeDonnees IdentificateurDePropriete {
    get {
        ...
        return valeurDeLaPropriete;
    }
    set {
        ... = value;
    }
}

```

Notez que l'on peut déclarer une propriété sans getter ou sans setter.

Par convention, on donnera toujours le même nom à la propriété et au champ privé qu'elle cache, en faisant commencer le nom du champ par une minuscule et celui de la propriété par une majuscule.

Les propriétés allègent le code et résolvent les problèmes de nommage, mais elles opacifient le code avec des appels de méthodes implicites.

En générale les propriétés ont une visibilité publique, leur but étant de fournir une interface vers l'extérieur de la classe. Néanmoins, on peut modifier individuellement la visibilité des accesseurs. Par exemple, pour avoir une propriété en lecture/écriture pour la classe mais en lecture seule pour les autres classes :

```

private double prix ;
public double Prix {
    get {...} // getter public
    private set {...} // setter privé
}

```

Dans le cas où une propriété n'a rien d'autre à faire que de retourner ou affecter directement la valeur d'un champ (sans modification, ni contrôle), on peut utiliser une forme simplifiée dite **propriété auto-implémentée**.

Ainsi la définition:

```

public double Prix { get ; private set ; } // propriété auto-implémentée avec modificateur privé

```

est équivalente à (à la différence qu'aucun champ `prix` n'est accessible même en privé):

```
private double prix ;
public double Prix {
    get { return prix ; }
    private set { prix = value ; }
}
```

Note

Il n'est pas toujours évident de choisir le bon type de membre pour ajouter une nouvelle fonctionnalité dans une classe : attribut ? méthode ? propriété ? constructeur ?...

Les [recommandation de conception](#) du guide de programmation C# peuvent vous aider à choisir l'approche la plus appropriée.

Exercice 10 : Propriétés

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

- -
Une propriété peut être de visibilité `public`.
- -
Une propriété peut être de visibilité `private`.
- -
Une propriété est toujours associée à un champ.
- -
Une propriété doit posséder un `get`.
- -
Une propriété doit posséder un `set`.
- -
Dans le setter d'une propriété, le mot clef `value` désigne la nouvelle valeur à affecter.
- -

Dans le getter d'une propriété, il n'est pas obligatoire de retourner une valeur.

- -

Dans le getter d'une propriété, je n'ai pas le droit de modifier les valeurs des champs de la classe.

[Corriger](#)

Exercice 11 : Utilisation des propriétés

Une classe `A` contient le membre suivant : `public string Name {get; set;}.`

Pour chacune de ces instructions, qu'on suppose écrite dans une méthode de la classe `A`, indiquez si elle est correcte ou non:

- -

```
Name.set("foo");
```

- -

```
Name.Set = "foo";
```

- -

```
Name.set = "foo";
```

- -

```
Name = "foo";
```

- -

```
SetName("foo");
```

- -

```
set("foo");
```

[Corriger](#)

Exercice 12 : Effet de bord

Une classe possède une propriété `P`, si je lis 2 fois de suite la valeur de cette propriété, j'obtiendrai :

- -
deux fois la même valeur
- -
deux valeurs différentes
- -
on ne peut pas savoir

Corriger

Membres statiques

Le qualificateur `static` permet de déclarer des **membres de classe**, aussi appelés **membres statiques**, qui ne dépendront pas d'une instance particulière de la classe. Par exemple la fonction racine carrée, la constante, la fonction `Main` (première fonction exécutée du programme) ne dépendent pas d'un objet particulier; elles existent même si aucun objet n'est créé.

```
class A{  
    public static int x;           // champ statique  
    public static void Bar(){};    // méthode statique  
    public static double UnNombre {get; set;} // propriété statique  
}
```

Les champs de classe sont initialisés avant la première utilisation de la classe où ils sont définis.

Les membres de classes peuvent être appelés n'importe quand et n'importe où dans votre programme même si aucun objet du type concerné n'a été créé. Pour accéder à un champ statique ou à une méthode statique, on utilise l'opérateur d'accès `.` préfixé par le nom de la classe (contrairement aux membres d'instance que l'on préfixe par la référence sur un objet). Lors de l'appel à une méthode statique il n'y a donc pas de référence vers une instance courante : le mot clef `this` n'est pas défini. Il n'est donc pas possible d'accéder aux membres d'instances.

Un exemple classique d'utilisation d'un champ statique est la définition d'un compteur d'instances créées pour une classe:

```
class A
{
    double valeur;
    public static int compteur = 0; // nombre d'objets de type A créés

    public A(double valeur){
        this.valeur = valeur;
        A.compteur++; // on vient de créer un nouvel objet
    }

    public static void Test() { // une méthode statique
        A a1 = new A(3.1);
        WriteLine(A.compteur); // 1
        A a2 = new A(3.2);
        WriteLine(A.compteur); // 2
        WriteLine(a1.valeur); // 3.1 accès à un champ d'instance à partir
d'une référence
//WriteLine(this.valeur); ERREUR la référence this n'est pas défini
dans une méthode statique
//WriteLine(a1.compteur); ERREUR accès à partir une référence à un
champ statique
    }
}
```

À la fin de l'exécution de la méthode `Test`, on peut représenter la mémoire de la façon suivante :

Afin de simplifier l'écriture, on peut accéder aux membres statiques de la classe courante sans utiliser le nom de la classe et l'opérateur `.`. Autrement dit, si on ne précise pas de nom de classe avec l'opérateur `.` devant un nom d'identificateur qui correspond à un membre statique, le compilateur ajoutera automatiquement `NomDeLaClasse.` devant l'identificateur. Le code suivant est donc valide:

```
class A
{
    public static int valeur;

    public void Test()
    {
        valeur = 3; // équivalent à A.valeur = 3
    }
}
```

Note

Le mot **statique** est associé à tout ce qui est calculé au moment de la compilation. Son opposé est le mot **dynamique** qui désigne ce qui est calculé pendant l'exécution du programme.

Exercice 13 : Human VM

```

class A{
    int i = 3;
    static int si = 2;

    void M(){
        i *= 2;
    }

    void static SM(){
        si *= 2;
    }

    void Test()    {
        M();
        SM();
        WriteLine(i);
        WriteLine(si);
    }

    static void Main(string [] args)    {
        A a1 = new A();
        A a2 = new A();
        a1.Test();
        a2.Test();
    }
}

```

Après exécution de la méthode `Main`, ce programme a affiché les 4 lignes de texte suivantes :

- 1.
- 2.
- 3.
- 4.

[Corriger](#)

Exercice 14 : Membres statiques

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

.....

Une classe peut contenir plusieurs membres statiques.

.....

On peut accéder à un membre statique d'une classe sans avoir créé d'objet de cette classe.

.....

Une méthode statique peut accéder à un champ d'instance à travers la référence de l'objet courant `this`.

Une méthode d'instance peut accéder à un champ statique à travers la référence de l'objet courant `this`.

On peut initialiser un champ statique en utilisant les valeurs des champs d'instances.

On peut initialiser un champ d'instance en utilisant les valeurs des champs statiques.

[Corriger](#)

Exercice 15 : Catégories de variables

```
class B{
    static double a;
    int b;
    void F(){
        float c;
    }
}
```

Pour chacune des variables `a`, `b` et `c` dites si il s'agit d'une variable d'instance et/ou de classe et/ou locale.

Variable	Instance	Classe	Locale
<code>a</code>	-	-	-
<code>b</code>	-	-	-
<code>c</code>	-	-	-

[Corriger](#)

Exercice 16 : Statique et règles d'accès aux champs

```
class A{
    public static int a;
```

```

        public int b;
    }

    class B{
        public static void Main(){
            A.a++;
            A.b++;
            A obj= new A();
            obj.a++;
            obj.b++;
        }
    }

```

- -
La ligne 8 est correcte.
- -
La ligne 9 est correcte.
- -
La ligne 11 est correcte.
- -
La ligne 12 est correcte.

[Corriger](#)

Exercice 17 : Statique et règles d'accès aux champs

```

class A{
    static int a;
    int b;

    public void F1(){
        a++;
        b++;
    }

    public static void F2(){
        a++;
        b++;
    }
}

```

- -
La ligne 6 est correcte.

- -

La ligne 7 est correcte.

- -

La ligne 11 est correcte.

- -

La ligne 12 est correcte.

[Corriger](#)

Exercices

Exercice 18 : Human VM

Soit la classe `C` et la fonction `Test`

```
class C
{
    public int a, b;
    public C bb;
    public C(int _a, int _b) { a=_a; b=_b; }
}

...

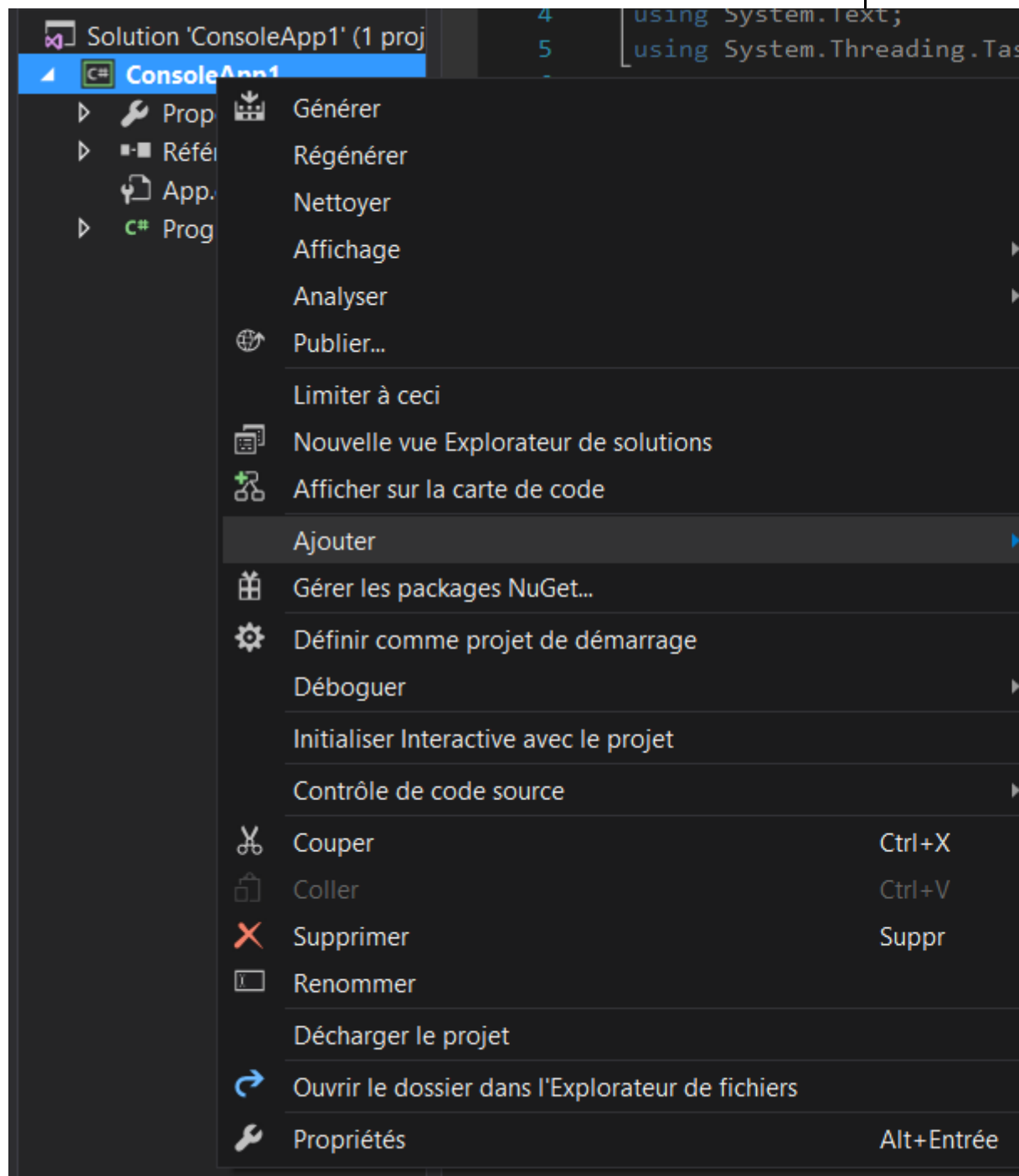
Test()
{
    C[] T = new C[5];
    for (int i = 0 ; i < 5 ; i++)
    {
        T[i] = new C(i,i);
        i++;
    }
}
```

Dessinez l'état de la mémoire à la fin de la fonction `Test` (un papier et un crayon reste la façon la plus simple de dessiner !)

Exercice 19 : Création d'une classe

Sous Visual Studio, créez un nouveau projet de type Console:

1. Créez une classe `Rectangle` : cliquez droit sur le nom du projet dans l'explorateur de solution, menu *Ajouter* -> *Classe*, renseignez le nom de la classe et cliquez sur OK.



2. Ajoutez des attributs à la classe : sa largeur `largeur`, sa hauteur `hauteur` en pixel et sa position à l'écran, `positionX` et `positionY`, en pixel.
3. Ajoutez un constructeur paramétrique qui initialise la taille et la position de l'objet.

4. Ajoutez une méthode d'instance `Aire` (sans argument) qui retourne l'aire du rectangle.
5. Dans la méthode `Main` de la classe `Program` :
 - A. Créez une nouvelle instance de la classe `Rectangle` de dimension 20 par 30 à la position (15,20).
 - B. Affichez les valeurs des attributs de l'objet créé.
 - C. Affichez l'aire de l'objet créé (utilisez la méthode `Aire`) afin de la calculer.
 - D. Exécutez votre programme et vérifiez que tout fonctionne correctement (ajoutez l'instruction `Console.Read();` à la fin du `Main` pour éviter que la fenêtre ne se ferme tout de suite).
6. Ajoutez un constructeur sans argument qui initialise la forme comme un carré de coté 10 et de position (0,0). Respectez le principe DRY : utilisez le chainage de constructeurs pour que le corps de votre constructeur ne contienne aucune instruction !
7. Ajoutez une méthode `DoublerTaille` qui multiplie la largeur et la longueur du rectangle par 2.
8. Dans la méthode `Main` de la classe `Program` :
 - A. Créez une nouvelle instance de la classe `Rectangle` avec votre constructeur sans argument.
 - B. Doublez la taille du rectangle créé avec la méthode `DoublerTaille`.
 - C. Affichez les dimensions du rectangle pour vérifier que tout a bien fonctionné.
9. Ajoutez une méthode `TestePlusPetit` qui prend en paramètre un rectangle `r` et retourne vrai si l'aire de `r` est plus petite que l'aire de l'instance courante.
10. Dans la méthode `Main` de la classe `Program` :
 - A. Testez la méthode `TestePlusPetit` avec les 2 références de `Rectangle` déjà créées.
11. Créez le constructeur de copie : constructeur qui reçoit en paramètre une référence sur un objet du même type et qui initialise à l'identique l'objet en cours de création. Respectez le principe DRY : utilisez le chainage de constructeurs pour que le corps de votre constructeur ne contienne aucune instruction !

12. Dans la méthode `Main` de la classe `Program` :

- A. Créez une copie du premier rectangle créé.
- B. Modifiez la largeur de la copie.
- C. Vérifiez que la largeur du premier rectangle n'a pas changé.

Exercice 20 : Human VM

Soit la classe `C` et la fonction `Test`

```
class C
{
    public int a,b;
    public C(int _a, int _b) { a=_a; b=_b; }
}

...

Test()
{
    C[] T = new C[5];
    for (int i = 0 ; i < 5 ; i++)
    {
        T[i] = new C(i,i+1);
    }
    T[2] = T[4];
    for (int i = 0 ; i < 5 ; i++)
    {
        T[i].a += T[i].b;
    }
}
```

Dessinez l'état de la mémoire à la fin de la fonction `Test` (un papier et un crayon reste la façon la plus simple de dessiner !)

Exercice 21 : Copie profonde

Une pratique courante est de proposer dans chaque classe un constructeur par copie, c'est-à-dire un constructeur qui prend en paramètre un objet du même type que celui à construire et qui construit une **copie profonde (deep copy)** de cet objet. Pour réaliser une copie profonde d'objet, il faut non seulement dupliquer l'objet mais également réaliser une copie profonde de tous les objets référencés par cet objet : l'objet d'origine et l'objet copié sont complètement indépendants. On parle de clonage.

Téléchargez et ouvrez cette [solution Visual Studio](#).

1. Ajoutez à la classe `Point` un constructeur prenant en paramètre un objet de type `Point` et qui construit une copie profonde de ce point.
2. Ajoutez à la classe `Polygone` un constructeur prenant en paramètre un objet de type `Polygone` et qui construit une copie profonde de ce polygone.

3. Décommentez le contenu du `Main` et exécutez le programme pour vérifier que tout fonctionne.

Donnez l'état de la mémoire à la fin de la fonction suivante (un papier et un crayon reste la façon la plus simple de dessiner !):

```
Point p1 = new Point(1,1);
Point p2 = p1;
Point p3 = new Point(p1);
Point [] pts = {p1, p2, p3};
Polygone po1 = new Polygone(pts);
Polygone po2 = new Polygone(pts);
Polygone po3 = new Polygone(po1);
p1.x=2;
p2.x=3;
p3.x=4;
po1.points[0].y=5;
po2.points[1].y=6;
po3.points[2].y=7;
```

Exercice 22 : Propriétés

Dans un nouveau projet Console sous Visual Studio, écrivez une classe `Duree` qui devra stocker une durée avec une précision à la seconde. La classe devra posséder 2 propriétés `DureeSeconde` et `DureeMinute` qui permettent de lire et modifier la durée avec des valeurs exprimées respectivement en secondes et en minutes. Une durée ne sera jamais négative. Pour exprimer une durée en minutes, on arrondira toujours à la valeur supérieure.

Exemple d'utilisation:

```
Duree d = new Duree(70); // durée de 70 secondes
WriteLine(d.DureeSeconde); // 70
d.DureeMinute = -2;
WriteLine(d.DureeSeconde); // 70
WriteLine(d.DureeMinute); // 2
d.DureeMinute = 2;
WriteLine(d.DureeSeconde); // 120
```

Exercice 23 : Course de lapin

Dans un nouveau projet Console sous Visual Studio:

1. Créez une classe `Lapin` qui contient les propriétés suivantes: `Surnom` (`String`), `Age` (`int`), `Position` (`int`). Vous implémenterez complètement les propriétés `Surnom` et `Age` et vous utiliserez une propriété auto-implémentée pour `Position`.
2. Ecrivez un constructeur permettant d'initialiser les propriétés `Surnom` et `Age` (`Position` sera toujours initialisée à 0).

3. Ajoutez un champ statique `nombreDeLapins` qui compte le nombre de lapins créés. Modifiez le constructeur pour que ce champ soit automatiquement incrémenté lors de la création d'un nouveau lapin.
4. Ajoutez une méthode statique `NombreDeLapins()` qui retourne le nombre lapins créé depuis le lancement du programme.
5. Ajoutez une méthode `void Avancer()` à la classe `Lapin`. Cette méthode ajoute une valeur aléatoire comprise entre 0 et 5 à la position du lapin. La méthode d'instance `Next` de la classe `Random` génère un nombre (pseudo) aléatoire.
6. Dans la méthode `Main` de la classe `Program`, créez trois lapins avec des noms et des ages différents.
7. Créez un tableau de lapins et rangez-y les trois lapins créés précédemment.
8. Appelez la méthode `Avancer()` 100 fois sur chacun des 3 lapins.
9. Le lapin qui a gagné est celui qui a la position la plus grande à la fin. Affichez le nom du lapin qui a gagné.

© Droits d'auteur B. Perret, ESIEE Paris 2022.

Compilé avec [Sphinx](#) en utilisant un [thème](#) fourni par [Read the Docs](#).

P00 et C#

- [Introduction à l'environnement .net](#)
- [Base du langage C#](#)
- [Les classes](#)
- [L'héritage](#)
- **[Compléments de langage](#)**
 - [Exceptions](#)
 - [Enumérations](#)
- [Fonctions avancées](#)
- [Projet - Space Invaders](#)
- [_____](#)
- [Git](#)
- [Space Invaders - Piste Verte](#)

-

- Compléments de langage
-

Compléments de langage¹³

Exceptions³

Une exception désigne une erreur pouvant se produire lors de l'exécution d'un programme.

En C# il existe deux sources d'exception :

- les exceptions systèmes (runtime): division par zéro, accès en dehors d'un tableau, utilisation d'une référence `null`. Ces exceptions sont le résultat d'une erreur de programmation (bug).
- les exceptions générées volontairement par le programmeur pour signaler un problème.

Historiquement, les exceptions étaient gérées par le programmeur au moyen de code d'erreur (par exemple en C): cette approche est lourde et manque de fiabilité, une stratégie plus systématique et gérée directement par le compilateur a donc été proposée dans les langages modernes (C#, Java, C++...).

Le mécanisme de gestion des exceptions reposent sur les principes suivants:

- Toutes les fonctions peuvent **lever/lancer** une exception.
- Toutes les fonctions peuvent déclarer des **blocs de gestion d'exceptions**.
- Lorsqu'une exception est levée, toutes les fonctions de la pile d'exécution sont terminées jusqu'à en trouver une capable de gérer cette exception. Si une telle fonction existe, l'exécution reprend dans le bloc de gestion d'exception de la fonction trouvée.
- Si aucune fonction n'est capable de gérer l'exception le programme est arrêté.

Considérons par exemple ce qui pourrait se produire lors de la sauvegarde d'une image dans un logiciel de dessin. Lorsque l'utilisateur clique sur le bouton *sauvegarder*, la fonction `SauvegarderIHM` est appelée. Cette fonction affiche une boîte de dialogue pour que l'utilisateur choisisse un fichier. Lorsque l'utilisateur a choisi un fichier de sauvegarde, le programme appelle la fonction `SauvegarderImage`. Cette fonction va convertir l'image dans un format standard (par exemple jpg) puis va appeler la fonction `EcrireFichier` qui permet d'écrire des données dans un fichier. Supposons maintenant que le disque dur de l'utilisateur soit plein. La fonction `EcrireFichier` lève une exception pour prévenir du problème. Son exécution est stoppée et le CLR regarde si la fonction qui l'avait appelée, c'est-à-dire `SauvegarderDocument` sait gérer l'exception. A priori cette fonction ne peut rien faire pour résoudre le problème et ne la gèrera pas. Le système remonte ensuite à la fonction `SauvegarderIHM`, qui elle peut gérer l'exception en demandant à l'utilisateur de choisir un autre emplacement de sauvegarde.

Avec un tel mécanisme il n'est pas possible d'interrompre (non intentionnellement) la propagation d'une exception.

Gérer une exception³

Pour gérer une exception on utilise les blocs **try/catch**. Le bloc **try** (essayer) comporte le code susceptible de générer des exceptions et le bloc **catch** (attraper) contient le code de gestion des exceptions.

```
int a,b,c;
...
try { // bloc try
    a = b/c ;           // erreur division par 0 si c==0
}
catch (Exception e){ // bloc catch
    Console.WriteLine("Erreur! {0}", e.Message);
}
```

Le bloc catch récupère un objet de type `Exception` qui contient une Propriété `Message` qui décrit l'exception.

Il existe différentes classes d'exceptions qui descendent de la classe `Exception` et il est possible d'affecter des blocs de traitement différents pour chaque exception.

```
int a,b,c;
int [] t;
...
try {
    t[a] = b/c ;
```

```

}
catch (AritmeticException e){
    Console.WriteLine("Erreur sur une opération arithmétique! {0}", e.Message);
}
catch (IndexOutOfRangeException e){
    Console.WriteLine("Accès en dehors des bornes d'un tableau! {0}", e.Message);
}
catch (Exception){
    Console.WriteLine("Erreur!");
}

```

Il y a donc des exceptions plus ou moins spécialisées, ce qui permet de ne gérer que certaines exceptions. Le type `Exception` désigne n'importe quelle exception et attrapera donc toutes les exceptions possibles.

Attention car l'ordre des `catch` est important pour savoir quel bloc va gérer l'exception. Par exemple, dans le code :

```

try {
    ...
}
catch (Exception){
    Console.WriteLine("Erreur sur une opération arithmétique!");
}
catch (IndexOutOfRangeException){
    Console.WriteLine("Accès en dehors des bornes d'un tableau");
}

```

Le bloc `catch` lié à l'exception `IndexOutOfRangeException` ne sera jamais atteint car le bloc précédent attrapera toutes les exceptions possibles.

Tous les objets décrivant une `Exception` possèdent les propriétés suivantes:

- `Message` : description de l'exception
- `StackTrace` : état de la pile d'exécution (liste des fonctions qui ont menées à cette exception)
- `Source` : nom de l'application ou de l'objet qui a levé l'exception
- `TargetSite` : nom de la méthode qui a levé l'exception
- `InnerException` : exception à l'origine de l'exception (si non null)

Lancer une exception [↗](#)

On lance une exception avec le mot clé **throw** suivi d'un objet de type `Exception` (ou un descendant). Par exemple:

```

double Racine(double a){
    if(a<0)
        throw new ApplicationException("Erreur : la racine carrée d'un nombre négatif n'existe pas.");
    return Math.Sqrt(a);
}

```

```
}
```

Bloc finally

Chaque bloc `try` peut être associé à un bloc **finally**. Les instructions placées dans le bloc `finally` seront exécutées même si une exception se produit dans le bloc `try`, que cette exception soit gérée ou non. Ce bloc est très utile pour gérer la libération des ressources tenues. Par exemple:

```
StreamWriter sw = null; // pour écrire dans un fichier
try {
    sw = new StreamWriter("y:\\monfichier.txt"); // ressource système vers un fichier
    sw.WriteLine("toto");
} catch (Exception e) {
    Console.WriteLine("Erreur : {0}", e.Message);
} finally { // bloc finally
    // ce block est toujours exécuté : c'est le bon endroit pour fermer le fichier !
    if (sw != null) // si la construction du StreamWriter a échoué alors sw est null :
        // le fichier n'a jamais été ouvert
        sw.Close(); // fermer le fichier
}
```

On peut même avoir un bloc `finally` sans bloc `catch`:

```
StreamWriter sw = null;
try {
    sw = new StreamWriter("y:\\monfichier.txt");
    sw.WriteLine("foo");
} finally {
    if (sw != null)
        sw.Close();
}
```

Exercices

Exercice 1 : Exceptions

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

Une méthode doit gérer les exceptions qui peuvent se produire dans les instructions qu'elle utilise.

Une exception non gérée sera ignorée par le programme.

Certaines exceptions ne sont pas causées par des bug de programmation.

L'expression `1/0` génère une exception.

L'expression `1.0/0.0` génère une exception.

Le bloc `finally` est optionnel après un bloc `try/catch`.

Le bloc `catch` doit apparaître au moins une fois après un bloc `try`.

Après une instruction `throw`, la fonction reprend son exécution normalement si l'exception a été gérée par une autre fonction.

Corriger

Exercice 2 : Human VM

```
class Test{
    void H(bool f){
        Console.Write("H1");
        if(f)
            throw new Exception("e1");
        Console.Write("H2");
    }

    void G(bool f){
        Console.Write("G1");
        try{
            Console.Write("G2");
            H(f);
            Console.Write("G3");
        } catch (IndexOutOfRangeException e)
        { Console.Write("G4"); }
        finally
        { Console.Write("G5"); }
        Console.Write("G6");
    }
}
```

```

void F(bool f){
    try{
        Console.WriteLine("F1");
        G(f);
        Console.WriteLine("F2");
    } catch (Exception e) { Console.WriteLine("F3"); }
    finally { Console.WriteLine("F4"); }
    Console.WriteLine("F5");
}
}

```

L'appel à la méthode `F(false)` affiche la chaîne de caractères

L'appel à la méthode `F(true)` affiche la chaîne de caractères

[Corriger](#)

Exercice 3 : Human VM

```

class Test{
    int f(int x){
        if(x<0)
            throw new ApplicationException("");
        Console.WriteLine("fin f ");
        return 1;
    }

    int g(int x){
        int res= f(x) + 1;
        Console.WriteLine("fin g ");
        return res;
    }

    public static void Main(String [] args) {
        try{
            int res=g(-1);
            Console.WriteLine("try " + res + " ");
        } catch (ApplicationException e1) {
            Console.WriteLine("catch E1 ");
        } catch (Exception e2) {
            Console.WriteLine("catch E2 ");
        } finally {
            Console.WriteLine("finally");
        }
    }
}

```

Ce programme affiche la chaîne de caractères

[Corriger](#)

Exercice 4 : Human VM

```

class A{
    static int a=10;
    static int [] t = new int[3];
    static int i, j;

    static void F(){
        try{

```

```

        Console.WriteLine("F1");
        t[i]=a/j;
        Console.WriteLine("F2");    }
    catch(ArithmeticException) { Console.WriteLine("F3");    }
    finally{ Console.WriteLine("F4");    }
    Console.WriteLine("F5");
}

static void G(){
    try{
        Console.WriteLine("G1");
        F();
        Console.WriteLine("G2");    }
    catch(ArithmeticException) { Console.WriteLine("G3");    }
    catch(IndexOutOfRangeException) { Console.WriteLine("G4");    }
    finally { Console.WriteLine("G5");    }
    Console.WriteLine("G6");
}

static void Main(String args){
    i=1;    j=1;
    G();
    i=1;    j=0;
    G();
    i=3;    j=1;
    G()
}

```

Le premier appel à la fonction `G` affiche la chaîne de caractères .

Le deuxième appel à la fonction `G` affiche la chaîne de caractères .

Le troisième appel à la fonction `G` affiche la chaîne de caractères .

[Corriger](#)

Enumérations

Une énumération est un type de données dont les valeurs sont données par une liste de noms symboliques. Imaginons que l'on veuille représenter l'état civil d'une personne qui peut prendre sa valeur parmi : *Célibataire*, *Marié*, *Divorcé* ou *Veuf*. Une solution simple consiste à définir des constantes numériques représentant ces différentes possibilités :

```

public const byte CELIBATAIRE = 0; // const signifie que la variable est une constante
public const byte MARIE = 1;
public const byte DIVORCE = 3;
public const byte VEUF = 4;

```

Mais cette solution est risquée car un état civil est représenté par un byte et rien n'empêche le programmeur d'y mettre une valeur qui ne représente aucun état civil valide (par exemple 8).

Les énumérations fournissent une solution élégante à ce problème. Elles permettent de définir un nouveau type `EtatCivil` qui prend ses valeurs parmi la liste de valeurs *Célibataire*, *Marié*... Cette définition est réalisée grâce au mot clef **enum** suivi de l'identificateur de l'énumération, puis de l'ensemble des valeurs qu'elle peut prendre mises entre accolades et séparées par des virgules.

```
enum EtatCivil {Celibataire, Marie, Divorce, Veuf} // définition du type EtatCivil
...
EtatCivil ec ; // définition d'une variable de type EtatCivil
ec = EtatCivil.Marie ; // modification de la valeur
```

On accède aux valeurs de l'énumération comme aux champs statiques d'une classe : `IdentificateurEnumeration.Valeur`. La valeur par défaut d'une variable d'un type énumération correspond au premier élément de la liste des valeurs.

Exemple d'utilisation dans un switch pour déterminer le nombre de parts intervenant dans le calcul de l'impôt sur le revenu :

```
EtatCivil ec = ... ;
...
double nombreDeParts = 0 ;
switch (ec)
{
    case EtatCivil.Celibataire :
        nombreDeParts = 1 ;
        break ;
    case EtatCivil.Marie :
        nombreDeParts = 2 ;
        break ;
    case EtatCivil.Divorce :
        nombreDeParts = 1 ;
        break ;
    case EtatCivil.Veuf :
        nombreDeParts = 1.5 ;
        break ;
}
```

Exercice 5 : Jeux de cartes

On souhaite écrire des classes permettant de représenter un jeu de cartes à jouer. Un tel jeu est construit de la manière suivante : Le jeu comprend 52 cartes à jouer, réparties suivant:

- treize valeurs : As, Roi, Dame, Valet, et les points (2, 3, 4, 5, 6, 7, 8, 9 et 10) ;
- quatre enseignes : pique, coeur, carreau et trèfle.

Dans un nouveau projet Console sous Visual Studio:

1. Écrivez une énumération `Enseigne` qui permet de représenter les 4 enseignes : Pique, Coeur, Carreau et Trèfle.
2. Écrivez une énumération `Valeur` qui permet de représenter les 13 valeurs : As, Roi, Dame, Valet, Deux, Trois, Quatre, Cinq, Six, Sept, Huit Neuf, Dix.
3. Écrivez une classe `Carte`. Cette classe doit posséder 3 propriétés et 1 constructeur :
 - Une propriété `EnseigneCarte` de type `Enseigne` avec un getter et un setter publics qui permettent de connaître et modifier l'enseigne de la carte ;
 - Une propriété `ValeurCarte` de type `Valeur` avec un getter et un setter publics qui permettent de connaître et modifier la valeur de la carte ;
 - Une propriété `Points` de type entier avec un getter qui donne le nombre de points de la carte. On considère que le nombre de points des cartes de valeurs As, Roi, Dame et Valet est égal à 10.
 - Un constructeur qui prend en paramètre une `Enseigne` et une `Valeur` et qui initialise les propriétés avec les valeurs des paramètres.
- Écrivez la classe `Jeu`. Cette classe possède :
 - Un champ privé de type tableau de carte nommé `cartes` ;
 - Un constructeur public sans paramètre qui remplit le tableau `cartes` avec les 52 cartes du jeu. Indication : il est possible de parcourir les éléments de l'énumération `Enseigne` avec une boucle `foreach` de la manière suivante :

```
foreach(Enseigne variable in Enum.GetValues(typeof(Enseigne))
```

© Droits d'auteur B. Perret, ESIEE Paris 2022.

Compilé avec [Sphinx](#) en utilisant un [thème](#) fourni par [Read the Docs](#).

P00 et C#

- [Introduction à l'environnement .net](#)
- [Base du langage C#](#)

- [Les classes](#)
- [L'héritage](#)
- [Compléments de langage](#)
- **[Fonctions avancées](#)**
 - [Surcharge d'opérateurs](#)
 - [Valeurs par défaut et arguments nommés](#)
 - [Arguments en nombre variable](#)
 - [Indexeurs](#)
 - [Délégués et fonctions anonymes](#)
- [Projet - Space Invaders](#)
- [Git](#)
- [Space Invaders - Piste Verte](#)

•

- Fonctions avancées

Fonctions avancées

Surcharge d'opérateurs

En C#, il est possible de redéfinir les opérateurs comme le signe `+` afin de pouvoir les utiliser avec d'autres types de données que ceux pris en charge par défaut (numériques et string). C'est un exemple de polymorphisme. Imaginons une classe `Vecteur`, sans redéfinition d'opérateur, on est obligé de passer par un appel de méthode pour additionner 2 vecteurs:

```
class Vecteur {
    double x, y;
    ...
    public Vecteur Add(Vecteur v)
    {
        return new Vecteur(x + v.x, y + v.y);
    }
}
```

```
...
Vecteur v1, v2, v3;
...
v3 = v1.Add(v2);
```

Avec la redéfinition d'opérateur on va pouvoir écrire les choses de manière plus intuitive:

```
class Vecteur {
    double x, y;
    ...
    public static Vecteur operator+(Vecteur v1, Vecteur v2)
    { return new Vecteur(v1.x + v2.x, v1.y + v2.y); }
}

Vecteur v1, v2, v3;
...
v3 = v1 + v2;
```

Seuls les opérateurs suivants peuvent être redéfinis:

- Opérateurs unaires: `+`, `-`, `!`, `~`, `++`, `--`
- Opérateurs binaires `+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`
- Opérateurs de comparaisons (binaires) `==`, `!=`, `<`, `>`, `<=`, `>=`

Une **redéfinition d'opérateur** est une **méthode statique** nommée **operator** suivie du **symbole de l'opérateur** à redéfinir qui doit respecter les règles suivantes :

- Opérateur unaire : la méthode prend un unique argument du type de la classe dans laquelle l'opérateur est redéfini et renvoi une valeur du même type.
- Opérateur binaire : la méthode prend deux arguments dont au moins un doit être du type de la classe dans laquelle l'opérateur est redéfini. Le type de retour est libre.

Notons que la commutativité des opérateurs n'est pas gérée par défaut. Par exemple, avec le code suivant qui redéfinit l'opérateur `*` entre un vecteur et un scalaire :

```
class Vecteur {
    double x, y;
    ...
    public static Vecteur operator * (Vecteur v1, double k)
    { return new Vecteur(v1.x * k, v1.y * k); }
}

...
Vecteur v1,v2;
v2 = v1 * 2.0; // ok Vecteur * double est défini
// v2 = 2.0 * v1; ERREUR : double * Vecteur n'est pas défini
```

Pour cela il faut redéfinir une deuxième fois l'opérateur `*` en inversant les arguments:

```
class Vecteur {
    ...
```

```
public static Vecteur operator * (double k, Vecteur v1)
{ return v1*k; } // redéfinition en utilisant la première définition
}
```

Exercice 1 : Redéfinition d'opérateurs

Dans la classe `A` (actuellement vide), quelles déclarations d'opérateur sont correctes:

```
static A operator++(A a){...}
```

```
static double operator++(A a){...}
```

```
static A operator+(A a1, A a2){...}
```

```
static double operator+(A a1, A a2){...}
```

```
static A operator+(A a1, double d){...}
```

```
static double operator+(A a1, double d){...}
```

Corriger

Valeurs par défaut et arguments nommés

Il est possible de donner des valeurs par défaut aux paramètres d'une fonction, il n'est alors plus nécessaire de donner une valeur à ces arguments lors de l'appel à la fonction. Cette approche peut remplacer avantageusement la définition de fonctions polymorphes dans certains cas.

```
int F(int a = 1 , int b = 0){  
    return a * a + b ;  
}  
  
F(2,3); // a = 2 , b = 3  
F(2); // a = 2, b = 0 (valeur par défaut pour b)  
F(); // a = 1 et b = 0 (valeurs par défaut pour a et b)
```

Les règles à respecter sont les suivantes :

- Tous les arguments sans valeur par défaut doivent être à gauche des arguments avec valeur par défaut.
- Il est interdit de sauter des arguments lors de l'appel de la fonction (on ne peut omettre que les derniers arguments).

Pour dépasser cette dernière limitation, on peut utiliser les arguments nommés. Les arguments nommés permettent d'appeler une fonction sans respecter l'ordre de déclaration des arguments en nommant explicitement les arguments:

```
F(a:2, b:3); // a = 2 , b = 3  
F(b:2, a:3); // a = 3 , b = 2  
F(a:2); // a = 2, b = 0 (valeur par défaut pour b)  
F(b:3); // a = 1, b = 3 (valeur par défaut pour a)
```

En cas d'ambiguïté sur la fonction à utiliser, c'est celle avec le moins d'argument déclaré qui est retenue.

```
public void F() { ... }  
public void F(int a =0) { ... }  
...  
F(); // appel la 1ère définition, signature (F).
```

Exercice 2 : Appels de fonction

La classe `A` possède une unique méthode statique `public static void G(int a, String s = "",int b = 1){}`, quelles sont les instructions correctes ?

`A.G();`

`A.G(2);`

`A.G(1, "a", 2);`

`A.G(1, "a");`

`A.G(, "a");`

`A.G(, "a", 2);`

`A.G(b:2, a:3);`

`A.G(a=1, b=3);`

Corriger

Arguments en nombre variable ³

Il est parfois impossible de connaître à l'avance le nombre exacte d'arguments d'une fonction. Par exemple supposons que l'on veuille écrire une fonction `Max` qui calcule le maximum d'un ensemble de nombres entiers. On pourrait définir par polymorphisme une version avec 2 arguments, 3 arguments, 4 arguments et ainsi de suite.

```
int Max(int a, int b) {...}
int Max(int a, int b, int c) {...}
int Max(int a, int b, int c, int d) {...}
```

Cela n'est bien entendu pas très efficace et limité. Une autre solution serait de définir une version basée sur un tableau d'entier.

```
int Max(int [] t) {...}
```

Mais on est alors obligé de mettre les nombres dans un tableau pour pouvoir utiliser la fonction:

```
int a, b, c, d;
...
d = Max(new int[]{a,b,c,d});
```

Le principe des arguments variables permet de conserver le meilleur des deux mondes. Pour déclarer une méthode avec un nombre d'arguments variable, on utilise le mot clé **params**. A l'intérieur de la méthode l'argument est alors vu comme un tableau mais la fonction peut être utilisée comme si il y avait effectivement un nombre variable d'arguments.

```
int Max(params int[] t){
    int m = Int32.MinValue;
    for (int v in t)
        if(v>m)
            m=v;
    return m;
}

Max(); // <=> max( new int[]{} ) tableau vide !
Max(1); // <=> max( new int[]{1} )
Max(1, 2, 3); // <=> max( new int[]{1,2,3} )
Max(3, 2, 4, 5, 3, 4, 3, 5, 3, 5); // ...
int [] t=...
Max(t);
```

Chaque méthode peut utiliser un seul argument de taille variable qui doit être le dernier argument déclaré.

```
void F(int a, string b, params char[] ct) { ... }
void G(params char[] ct, params string[] cs) { ... } // erreur 1 argument de longueur variable au plus
```

```
void H(params char[] ct, int a) { ... } // erreur L'argument de taille variable doit être en dernière position
```

Exercice 3 : Paramètres de fonction

Dans la classe `A` (actuellement vide), quelles déclarations de fonction sont correctes:

```
void F(int a=2, int b){}
```

```
int F(int b, float b=3){}
```

```
F(int c=1, double b=1.0){}
```

```
void F(params double [][] d){}
```

```
void F(params double d){}
```

```
void F(int a, params double [] d){}
```

```
void F(params double d, int a=1){}
```

Corriger

Exercice 4 : Résolution de nom

Soit le programme suivant:

```
class A{
    public static void F(){ } // 1ère
    public static int F(int i=0){return i;} // 2ème
    public static double F(int i, int j){return i;} // 3ème
    public static double F(int i, double j=0.0){return j;} // 4ème
    public static double F(double i, int j=1){return i;} // 5ème
```

```
public static void F(string b){} // 6ème
public static void F(string b, string a){} // 7ème
}
```

Pour chaque appel de fonction, indiquez quelle définition de `F` est utilisée ou « Erreur » si l'appel n'est pas valide.

Appel de fonction	Définition utilisée	Appel de fonction	Définition utilisée
<code>F(1)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème	<code>F("2", 1)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème
<code>F(2.0)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème	<code>F("", 2 + ".0")</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème
<code>F("" + 2.0)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème	<code>F(true)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème
<code>F(1, 2)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème	<code>F("" + false)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème
<code>F(1.0, 2)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème	<code>F()</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème
<code>F(1.0, 2.0)</code>	Erreur 1ère 2ème 3ème 4ème 5ème 6ème 7ème		

[Corriger](#)

Indexeurs

Un indexeur permet d'accéder aux éléments d'un objet comme si il s'agissait d'un tableau. Ainsi, si l'on dispose d'une classe `Matrice` (tableau 2D de double avec des opérations mathématiques), sans indexeur on écrit des choses du genre :

```
Matrice m = ...
m.SetValeur(1,2, m.GetValeur(0,1) + 2.2);
```

La même chose avec indexeur :

```
Matrice m = ...
m[1,2] = m[0,1] + 2.2;
```

Mais les indexeurs sont beaucoup plus souples que ce que proposent les tableaux car il est possible de définir des indexeurs sur autre chose que des entiers. Par exemple, on peut définir des tableaux relationnels (indicé sur autre chose que des entiers, `Dictionary` en C#):

```
TabRel t = ...  
t["foo"] = "bar";  
string s = t["truc"];
```

Pour définir un indexeur: il faut définir une propriété dont le nom est **this** suivi entre crochets du type de l'indexeur, la définition du getter et du setter est alors classique (bien sûr un indexeur ne peut pas être une propriété auto-implémentée).

Exemple : la classe Annuaire permet de d'associer des noms à des numéros de téléphone. Elle offre deux indexeurs, le premier à partir d'un nombre entier permet de récupérer ou modifier le numéro du i-ème contact, le second permet de chercher modifier un numéro à partir d'un nom.

```
class Contact {  
    public string nom, numero;  
}  
  
class Annuaire {  
    List<Contact> list;  
  
    public string this[int n]{ // indexeur sur des entiers  
        get { return list[n].numero ; }  
        set { list[n].numero = value ; }  
    }  
    public string this[string n]{ // indexeur sur des strings  
        get { return list[Find(n)].numero ; }  
        set { list[Find(n)].numero = value ; }  
    }  
  
    private int Find(string n){  
        for(int i=0;i<list.Count;i++)  
            if(list[i].nom.Equals(n))  
                return i;  
        return -1;  
    }  
}
```

Exercice 5 : Indexeurs

Pour chacune de ces affirmations, indiquez si elle est vraie ou fausse:

Un indexeur est défini avec le mot clé `indexer`.

Un indexeur peut être privé.

Un indexeur peut être statique.

Un indexeur est une propriété.

Un indexeur peut utiliser des nombre réels comme indices.

Une classe peut posséder plusieurs indexeurs.

Corriger

Exercice 6 : Tableau à indices décalés

En C# tous les tableaux sont indicés à partir de zéro. Néanmoins la possibilité de définir soit même des indexeurs permet de créer des objets mimant le comportement d'un tableau avec une plage d'indices décalée (commençant par exemple à -20 ou à 1).

On souhaite définir une classe `TabInt` représentant un tableau d'entiers avec des indices décalés qui s'utiliserait de la manière suivante:

```
TabInt t = new TabInt(11, -5); // tableau de 11 éléments commençant à l'indice -5
t[-5]=-5; // initialisation du 1er élément du tableau
for(int i=t.IStart+1; i<t.IStart+t.Length ; i++)
    t[i]=t[i-1]+i;
Console.WriteLine(t[t.IStart+T.Length-1]); // affichage du dernier élément t[5]
```

Qu'affiche ce programme ?

Dans un nouveau projet Console sous Visual Studio, définissez la classe `TabInt` contenant:

- un champs privé de type tableau d'entiers;
- une propriété `IStart` de type entier avec accès en lecture publique et accès en écriture privé qui représente l'indice de départ;
- une propriété `Length` de type entier en lecture seule qui envoie le nombre d'élément dans le tableau;

- un constructeur public prenant en argument : le nombre d'éléments dans le tableau et l'indice de départ;
- un indexeur permettant d'accéder aux éléments du tableau (lecture et modification) en prenant compte du décalage d'indice.
- une méthode public `SetAll` qui prend en arguments l'indice de départ et un nombre variable d'argument de type entier et qui place ces nombres dans le tableau à partir de l'indice indiqué. Par exemple:

```

• TabInt t = new TabInt(5, 1); // tableau de 5 entiers tous égaux à 0
•                               // commençant à l'indice 1
• t.SetAll(2,1,2,3); // Mets les éléments 1,2,3 à partir de l'indice 2
• // t contient maintenant 0 1 2 3 0

```

Délégués et fonctions anonymes [↗](#)

Un délégué est un objet qui permet d'appeler une ou plusieurs fonctions. L'intérêt étant qu'il est possible de changer dynamiquement la ou les fonctions qui seront appelées par le délégué. Les délégués sont énormément utilisés dans la programmation d'interface graphique qui consiste à associer des actions à des événements. Lors d'un clic de souris, le système consulte le délégué associé à cet événement et lui demande d'exécuter les fonctions que le développeur à assigner à cet événement.

Par exemple, dans le fichier `Form1.Designer.cs` (généré automatiquement par le designer de Visual Studio lors de la création d'une IHM Win32), après avoir créé un bouton et définit une action en cas de clic de l'utilisateur, on trouve:

```
this.button1.Click += new System.EventHandler(this.button1Click);
```

C'est-à-dire, une affectation de la propriété `Click` de la référence `button1`. La valeur affectée correspond à une référence vers un nouvel objet de type `EventHandler` (c'est un délégué) avec en paramètre la fonction `button1Click`.

Les délégués sont comparables (mais en mieux) aux pointeurs de fonctions du C/C++.

Type délégué [↗](#)

Un **délégué** est toujours **associé à un prototype de fonction** particulier qui sera le **type du délégué**. Ainsi, un délégué donné ne pourra référencer que des fonctions qui prennent les mêmes arguments et retournent le même type de donné.

La déclaration d'un type délégué se fait grâce au mot clef **delegate**. Par exemple:

```
delegate int TypeDelegate( double v );
```

déclare le type délégué nommé `TypeDelegate` et correspondant à une fonction prenant un paramètre de type `double` et retournant un `int`. On peut ensuite déclarer des objets de type `TypeDelegate` et leur assigner des fonctions. Puis l'objet de type délégué s'utilise comme une fonction:

```
static int F(double v){  
    return (int)(v+0.5);  
}  
  
static void Main(String [] args){  
    TypeDelegate d = new TypeDelegate(F); // d est un objet délégué pointant sur F  
    int i = d(2.7); // appel de F à travers le délégué d  
    Console.WriteLine(i); // 3  
}
```

On peut changer la fonction associée à un objet délégué en cours d'exécution:

```
static double F(double v){ WriteLine(++v); return v; }  
static double G(double v){ WriteLine(--v); return v; }  
  
delegate double T( double v );  
  
static void Main(string [] args)  
{  
    T d = new T(F); // d est un objet délégué pointant sur F  
    d(1.0); // 2  
    d = new T(G); // d est un objet délégué pointant sur G  
    d(1.0); // 0  
}
```

On peut également assigner des méthodes d'instance à un délégué:

```
class A {  
    int x=2;  
    public void F(){ WriteLine(x) ;}  
  
    delegate void T();  
  
    public static void main (string [] args)  
    {  
        A a = new A();  
        T d = new T(a.F);  
        d(); // affiche 2  
        a.x++;  
        d(); // affiche 3  
    }  
}
```

`d` référence la méthode d'instance `F` associée à l'objet référencé par `a` et accède à ses champs d'instance.

Un autre cas typique d'utilisation des délégués (en dehors des évènements) est la fonction `Map` qui sert à appliquer une fonction à tous les éléments d'une liste.

```
delegate double FonctionNumerique(double x); // type délégué sur des fonctions réelles

void Map ( double [] tab, FonctionNumerique f){ // une fonction qui prend une *fonction* en
paramètre
    for(int i=0; i<tab.Length; i++)
        tab[i] = f(tab[i]);
}

double [] t = {1.1,-2.8,3.7} ;
Map(t, new FonctionNumerique(Math.Round));
// t => {1.0,-2.0,3.0}
Map(t, new FonctionNumerique(Math.Abs));
// t => {1.0,2.0,3.0}
```

Fonctions anonymes

Les fonctions que l'on associe à un objet délégué sont rarement utilisées autrement qu'à travers ce délégué. Comme une fonction n'est jamais appelée directement, il est inutile de la nommer. On peut alors recourir à une déclaration compacte avec le mot clé **delegate**.

Exemple:

```
delegate int T (int n) ; // type délégué

T d = new T(
    delegate (int n){ return n++ ; } // fonction anonyme
);
```

Ce qui revient à la déclaration longue :

```
delegate int T (int n) ; // type délégué

int MaFonction(int n){
    return n++;
}

T d = new T(MaFonction);
```

Il est bien sûr possible de mettre plusieurs instructions dans le corps de la fonction anonyme, de déclarer des variables, d'appeler d'autres fonctions...

```
double[] t = {5,50,500} ;
Map(t, new FonctionNumerique(
    delegate (double x){
        double tmp = x*2 ;
        return Math.Log10(tmp) ;
    }
)); // t=> {1,2,3}
```

Expressions lambda

Les **expressions lambda** (vient de la notion de lambda calcul) sont une autre façon d'écrire des fonctions anonymes. Par exemple :

```
x => x + 10
```

est une fonction qui prend un paramètre `x` et renvoie la valeur de `x + 10`. Remarquez qu'il n'y aucune information de typage explicite.

On peut également mettre plusieurs instructions dans une lambda expression, dans ce cas il faut préciser le `return` :

```
x => {double y=x*x ; return Math.Log(y) ;}
```

ainsi que prendre plusieurs paramètres :

```
(a,b) => a<b  
(a,b) => {a++, b--, return a*b ;}
```

On peut maintenant simplifier les exemples précédents:

```
delegate int T (int n) ; // type délégué  
T d = n => n++;
```

Ou en reprenant l'exemple de la fonction Map :

```
double[] t = {5,50,500} ;  
Map(t, x=>{double tmp = x*2 ; return Math.Log10(tmp) ;} ) ; // t=> {1,2,3}
```

Exercices

Exercice 7 : QCM Délégués

Le mot clef `delegate` permet de déclarer un nouveau type de données.

Le mot clef `delegate` permet de déclarer une fonction anonyme.

Une variable de type délégué peut référencer une méthode statique.

Une variable de type délégué peut référencer une méthode d'instance.

Une fonction anonyme peut prendre plusieurs paramètres.

Une fonction anonyme peut être composée de plusieurs instructions.

Une fonction anonyme ne contient pas d'information de typage.

Une lambda expression peut prendre plusieurs paramètres.

Une lambda expression peut être composée de plusieurs instructions.

Une lambda expression ne contient pas d'information de typage.

Une lambda expression peut servir à initialiser une variable de type délégué.

Corriger

Exercice 8 : Délégués, premières manipulations

Dans un nouveau projet Console, dans la classe `Program` :

- Déclarez le type délégué `IntAction` qui prend un `int` en argument et ne retourne rien.
- Déclarez une méthode statique `PrintInt` qui prend un argument de type entier, ne retourne rien et affiche l'entier reçu en paramètre sur la console.

- Déclarez une méthode statique `PrintIntSquare` qui prend un argument de type entier, ne retourne rien et affiche le carré de l'entier reçu en paramètre sur la console.
- Déclarez une méthode statique `Perform` prenant comme paramètre un élément de type `IntAction`, un nombre variables d'éléments de type `int` et qui applique le délégué `IntAction` à chacun des éléments entiers au moyen d'une boucle `foreach`.
- Dans la méthode `Main`, réalisez les instructions suivantes:
 - Déclarez une variable `act` de type `IntAction`.
 - Affectez la méthode `PrintInt` à la variable `act`.
 - Appelez le délégué `act` avec la valeur 42.
 - Déclarez un tableau d'entiers `tab` contenant les valeurs `{1,2,3,4}`.
 - Appelez la méthode `Perform` avec pour paramètre la variable `act` et le tableau `tab`.
 - Appelez la méthode `Perform` avec pour paramètre la variable `act` et les valeurs `9,8,7,6,5` (sans déclarer de nouveau tableau).
 - Affectez la méthode `PrintIntSquare` au délégué `act`.
 - Appelez la méthode `Perform` avec pour paramètre la variable `act` et le tableau `tab` : qu'affiche le programme?